

DKEM

(Ding Key Encapsulation)

June 30, 2026

Submitted To
Institute of Commercial Cryptography Standards

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Contributions	5
1.3	Design Objectives	6
2	Algorithm Description	8
2.1	Preliminaries	8
2.1.1	Ring, Modules, and Notation	8
2.1.2	Noise Distributions	11
2.1.3	Public Parameters	12
2.1.4	Matrix Generation and Sampling Algorithms	13
2.2	Core Functions	14
2.2.1	Reconciliation Mechanism	14
2.2.2	Number-Theoretic Transform (NTT)	17
2.2.3	Compression and Decompression Functions	18
2.3	Specification of DKE	19
2.4	Specification of CCA-Secure DKEM	19
2.5	Parameter Sets	20
3	Design Rationale	22
3.1	Design goals and the ICCS context	22
3.2	The mathematical hard problem	23
3.2.1	The CPA reconciliation core (DKE)	24
3.2.2	CCA transform with tag-based Fujisaki-Okamoto	24
3.3	Criteria for Parameter Selection	24
3.3.1	Alignment with ML-KEM and Architectural Simplification	24
3.3.2	High-Security Parameter Set (512-bit)	26
3.4	Failure probability analysis.	26
3.5	Other considerations	28
4	Performance Evaluation	29
5	Security Statements and Analyses	33
5.1	Theoretical security	33
5.1.1	Security model and proof outline	34
5.1.2	The Module-LWE Assumption	34
5.1.3	Pseudorandomness of the DKE Shared Secret	35
5.1.4	IND-CCA security of DKEM	38

5.2	Analysis with respect to known attacks	42
5.2.1	Attacks against the underlying MLWE problem	42
5.2.2	Refined gate-count estimate	45
5.2.3	About quantum security	47
5.2.4	Attacks against symmetric primitives	48
5.2.5	Attacks exploiting reconciliation failures	48
5.3	Other security properties	49
6	Feature Statements	51
A	Technical Proofs	61
B	Parameter selection	64

1 Introduction

1.1 Background and Motivation

The advent of large-scale quantum computers threatens the security of public-key cryptosystems that are widely deployed today. In particular, schemes whose security is based on the hardness of integer factorization or the discrete logarithm problem can be efficiently broken using Shor’s algorithm [1]. This motivates the study and standardization of post-quantum cryptographic (PQC) primitives that remain secure in the presence of quantum adversaries.

Among the most prominent families of PQC schemes are those based on lattices, and, in particular, on the Learning With Errors (LWE) problem and its structured variants [2, 3, 4]. LWE and Module-LWE have been extensively studied from both the theoretical and practical perspectives and are now the foundation of several standardized or soon-to-be-standardized schemes. In the context of key establishment, LWE-based constructions typically follow a common pattern: parties exchange *noisy* algebraic objects that encode a shared secret, and then apply a *reconciliation* mechanism to derive identical bit strings from their respective noisy views.

Ding Key Exchange [5, 6] is the first and a representative example of such an LWE-based key exchange protocol. It operates over vector spaces $\mathbb{Z}_q^n$ and polynomial rings over $\mathbb{Z}_q$ as well, and uses carefully designed error distributions together with a reconciliation mechanism that allows two parties to agree on a shared key with high probability despite the presence of noise. The central idea is that both parties hold LWE-like samples which, after suitable algebraic operations, yield two close but not identical vectors. A deterministic rounding-based procedure, aided by a small amount of helper data, maps both vectors to the same bit string except with negligible failure probability.

In parallel, substantial progress has been made in the design and optimization of Module-LWE-based schemes over structured rings. In particular, the scheme historically known as Kyber [7] has been standardized by NIST as the Module-Lattice Key Encapsulation Mechanism (ML-KEM) and specified in FIPS 203 [8]. ML-KEM is defined over a small cyclotomic ring $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ with modulus $q = 3329$ and uses the Module-LWE problem over R_q^k , for small module ranks $k \in \{2, 3, 4\}$. Its design leverages the structure of R_q to achieve efficient arithmetic via the Number Theoretic Transform (NTT), compact keys and ciphertexts via compression, and well-understood security levels through a careful choice of parameters and noise distributions.

The success of ML-KEM demonstrates that Module-LWE over structured rings can meet stringent efficiency and implementability requirements while providing conservative security margins. At the same time, the conceptual clarity of Ding Key Exchange’s reconciliation-based interactive key exchange suggests that a synthesis of these two lines of work could yield a protocol that enjoys both the efficiency and implementation maturity of ML-KEM and the

transparent, interactive design philosophy of Ding Key Exchange.

1.2 Contributions

This work proposes a module-lattice-based Key Encapsulation Mechanism (KEM), named *DKEM*, adopting parameter choices and some implementation techniques of ML-KEM.

This KEM is derived (through a Fujisaki-Okamoto-like transform [9]) from a core **ephemeral key exchange protocol**, named **DKE**, which improves the Ding Key Exchange by incrementing the number of signal bits. This key exchange protocol is specified in Section 2.3.

Concretely, DKEM and DKE are instantiated over the same ring

$$R_q = \mathbb{Z}_q[X]/(X^{256} + 1), \quad q = 3329,$$

and use the same module ranks $k \in \{2, 4\}$ and secret/noise distributions as the ML-KEM parameter sets ML-KEM-512, and ML-KEM-1024, corresponding to NIST security levels 1 and 5 respectively, and NGCC levels of 128 and 256 classical security bits.

We also instantiate the protocol over

$$R_q = \mathbb{Z}_q[X]/(X^{512} + 1), \quad q = 7681,$$

and $k = 4$ in order to achieve the highest level of security of 512 classical security bits requested by NGCC.

At a high level, DKE can be viewed as a “module-lattice lift” of the original Ding Key Exchange protocol [5, 6] with an improved reconciliation mechanism. Vector spaces $\mathbb{Z}_q^n$ are replaced by modules R_q^k , and the public LWE matrix is replaced by a structured Module-LWE matrix $\mathbf{A} \in R_q^{k \times k}$ that is deterministically generated from a short public seed, exactly as in ML-KEM. The parties compute noisy shared values in R_q^k by multiplying the exchanged module elements with their secret vectors and adding small error terms. A reconciliation mechanism, improved from Ding Key Exchange and tailored to the noise bounds and modulus of ML-KEM, maps these noisy shared values to identical bit strings with probability comparable to ML-KEM’s and much smaller than the original Ding Key Exchange’s.

The proposed ephemeral scheme is defined in the form of an *interactive two-pass key exchange* protocol, in which an initiator A sends a first message containing an LWE-like sample, and a responder B replies with its own sample plus signal information, allowing both to reconcile errors and derive a shared session key.

By fixing the underlying ring, modulus, and noise distributions to those of ML-KEM, DKEM inherits much of ML-KEM’s security analysis and practical performance characteristics. At the same time, it showcases how Ding-style reconciliation can be naturally integrated into a Module-LWE setting and further improved to be more practical. The resulting construction is intended to be simple to specify, straightforward to implement using some of the existing

ML-KEM codebases, and amenable to rigorous security analysis under well-understood assumptions.

Moreover, the internal ephemeral DKE described in this protocol is useful by itself in some contexts in which key reuse is not a requirement. For instance, it has been proven in the Random Oracle Model (ROM) that CPA KEMs are suitable for applications such as TLS [10], offering several advantages in terms of bandwidth and performance with respect to their CCA-secure counterparts. For that reason, by using Post Quantum signatures, we can also evolve from DKE an authenticated key exchange, named **DKEX**, suitable for many contexts in the post quantum world. Moreover, inspired by the general construction described in [11], we have used DKEM to instantiate a one-sided authenticated key exchange named **ADKEX**. Both protocols are specified in parallel submissions to the NGCC project as Key Exchange protocols.

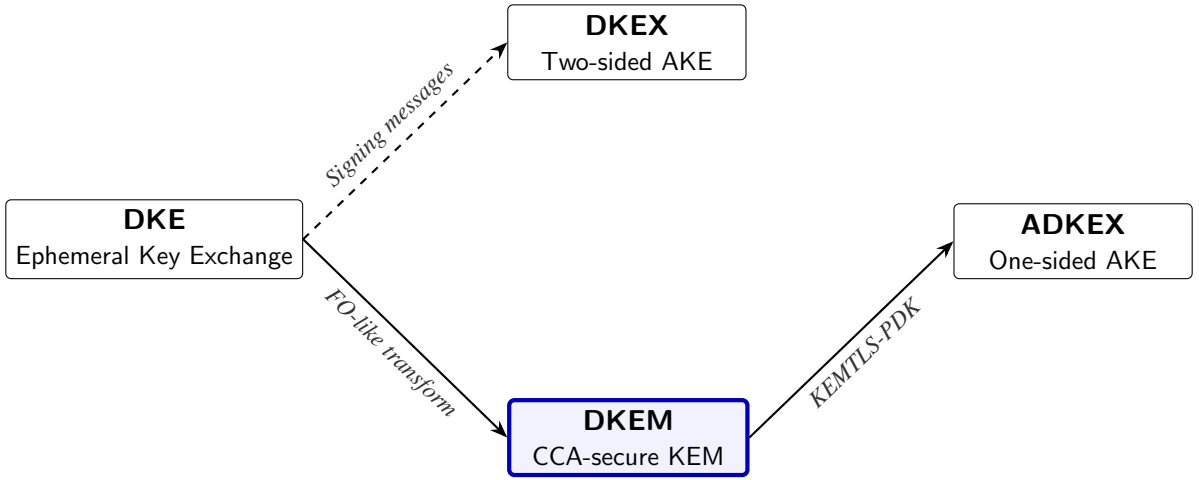


Figure 1: Key exchange scheme transformations.

1.3 Design Objectives

The development of DKEM is guided by several design objectives, aiming to deliver a highly secure, efficient, and easily deployable Key Encapsulation Mechanism:

Security. The primary objective is to obtain a key exchange protocol whose security can be reduced to the hardness of Module-LWE over the ring R_q . The scheme is designed to achieve IND-CPA security under the Module-LWE assumption, and IND-CCA security when combined with a Fujisaki–Okamoto (FO) [9] style transform.

Negligible failure probability. A primary advancement in this proposal is the deployment of a highly refined reconciliation mechanism utilizing fine-grained signal processing. This architectural enhancement effectively eliminates the higher failure rates observed in early Ding Key Exchange variants (e.g., 2^{-60}), driving the decryption failure probability down to cryptograph-

ically negligible bounds (e.g., $\leq 2^{-132.7}$). The key technique of this proposal is to improve the reconciliation mechanism by adopting more fine-grained signals.

Robust Algebraic Foundations and Ecosystem Compatibility as a guide for parameter selection. To provide the highest level of confidence in the concrete security of DKEM, the core algebraic parameters — the ring R_q , module rank k , and error distributions — are selected from the most rigorously scrutinized parameter sets in lattice cryptography [8]. By adopting these established foundations, DKEM guarantees conservative security levels while remaining completely aligned with the cryptographic community’s current expectations for Module-LWE parameters.

Efficiency and implementation maturity. DKEM is designed for efficient deployment across diverse platforms, ranging from high-performance servers to resource-constrained embedded devices. To achieve this, the scheme allows the direct reuse of heavily optimized, constant-time lattice primitives. By aligning its core algebraic operations with established parameters, such as those standardized in [8], DKEM implementations can seamlessly integrate with existing public-domain codebases. This alignment enables the direct utilization of rigorously scrutinized and verified Module-KEM routines from open-source cryptography projects, most notably PQClean [12], liboqs [13] or mlkem-native [14]. Reusing these tested repositories for NTT-based polynomial arithmetic, pseudo-random matrix generation, and secure sampling procedures significantly minimizes the engineering burden, accelerates deployment, and mitigates the risk of introducing novel implementation errors or side-channel vulnerabilities. We also implement DKEM using new more efficient techniques, such as an improved Plantard method for lattice-based cryptography [15].

Simplicity and modularity. The protocol is structured to be conceptually simple and modular. The separation between the Module-LWE core (matrix generation, secret/noise sampling, linear algebra over R_q) and the reconciliation layer (quantization and helper data) is made explicit. This separation facilitates independent analysis of noise growth and reconciliation correctness, and allows potential future variants to experiment with alternative reconciliation strategies without modifying the underlying Module-LWE structure.

Suitability for Standardization. DKEM specification aims to provide a comprehensive and transparent framework. By pairing a reconciliation layer with understood Module-LWE foundations, this proposal simplifies the formal evaluation process. Standardization bodies and independent cryptanalysts can conduct security and performance assessments within a familiar theoretical context. Rather than requiring the community to audit and engineer an entirely new cryptographic paradigm from scratch, DKEM offers a secure and simple path from specification

to real-world deployment. Reference and optimized implementations can be built from modifying and improving existing ML-KEM code and including new implementation techniques [15]. This alignment enables rapid prototyping, accelerates testing, and ensures that broad adoption can proceed with maximum confidence and minimal engineering overhead.

2 Algorithm Description

2.1 Preliminaries

In this subsection we describe the algebraic setting, define various notations and the noise distributions used throughout the scheme, fix the public parameters and notation, and describe the generation of the public matrix $\mathbf{A}$ from a seed.

2.1.1 Ring, Modules, and Notation

Base field and modulus. Let q be an odd prime. We write $\mathbb{Z}_q$ for the finite field with q elements, and we identify it with the set of integers $\{-\frac{q-1}{2}, \dots, 0, \dots, \frac{q-1}{2}\}$ when discussing magnitudes or norms. All additions and multiplications of coefficients are taken modulo q unless otherwise stated. In this specification, $q = 3329$ for the two firsts levels of security, and is incremented to $q = 7681$ for the last security level.

Polynomial ring. Consider the polynomial ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1),$$

where $n = 256$ for the first two security levels and $n = 512$ for the third. An element $a \in R_q$ is represented by a polynomial

$$a(X) = a_0 + a_1X + \dots + a_{n-1}X^{n-1} \quad \text{with } a_i \in \mathbb{Z}_q,$$

where polynomials are reduced modulo $X^n + 1$, and coefficient-wise modulo q . We will often identify a with the coefficient vector $(a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$.

Modules and matrices over R_q . For $k \geq 1$, we write R_q^k for the set of column vectors

$$\mathbf{v} = (v_1, \dots, v_k)^\top \quad \text{with } v_i \in R_q.$$

Matrices over R_q are denoted by bold uppercase letters: $\mathbf{A} \in R_q^{m \times k}$ is an $m \times k$ matrix whose entries are in R_q . Matrix-vector multiplication ($\mathbf{A}\mathbf{s} \in R_q^m$) and vector addition are defined in the natural way using ring operations in R_q .

Elements of R_q^k and $R_q^{m \times k}$ can be equivalently viewed as arrays of polynomials or as arrays of length- n coefficient vectors, depending on the context.

Inner products and norms. For $\mathbf{u}, \mathbf{v} \in R_q^k$, we define their R_q -valued inner product as:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^k u_i \cdot v_i \in R_q.$$

We also denote $\mathbf{u} \cdot \mathbf{v} = \langle \mathbf{u}, \mathbf{v} \rangle$.

When needed, we endow R_q and R_q^k with coefficient-wise norms. For $a \in R_q$ with coefficient vector $(a_0, \dots, a_{n-1})$ — lifted to integers in the canonical interval —, we define:

$$\|a\|_\infty := \max_{0 \leq i < n} |a_i|, \quad \|a\|_2 := \left(\sum_{i=0}^{n-1} a_i^2 \right)^{1/2}.$$

These norms extend naturally to vectors $\mathbf{v} \in R_q^k$ by viewing $\mathbf{v}$ as a length- kn integer vector.

Number Theoretic Transform (NTT). Efficient polynomial multiplication in R_q is achieved using the Number Theoretic Transform (NTT) and its inverse, as in [8]. Recall that $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, with n a power of two and q an odd prime. The arithmetic depends on the factorization of $X^n + 1$. At first it was thought that full splitting was needed, that is, $2n \mid (q-1)$, but it was proved that having quadratic splitting is as efficient and this significantly reduces the needed modulus [16]. We are on the quadratic splitting case in all our parameter sets.

Suppose that $n \mid (q-1)$ but $2n \nmid (q-1)$. In this case, $\mathbb{Z}_q$ contains a primitive n -th root of unity but not a primitive $2n$ -th root. Choose $\zeta \in \mathbb{Z}_q$ such that $\zeta^n = 1$, and let $r := \log_2(n/2)$.

We define the r -bit reversal map: `bit_rev`, for $n = 2^r$, which maps an unsigned r -bit value $m = \sum_{s=0}^{r-1} a_s 2^s$ to an integer as:

$$\text{bit_rev}_r(m) = \sum_{s=0}^{r-1} a_s 2^{r-1-s}, \quad a_s \in \{0, 1\}.$$

The following factorization can be expressed in bit reversal:

$$X^n + 1 = \prod_{i=0}^{n/2-1} (x^2 - \zeta^{2\text{bit_rev}_r(i)+1}).$$

For $(n, q) = (256, 3329)$, we have:

$$x^{256} + 1 = \prod_{i=0}^{127} (x^2 - \zeta^{2\text{bit_rev}_7(i)+1}).$$

In this case $\zeta = 7$ is taken, since it is the smallest of all possible values. Similarly, for (512, 7681) we have:

$$x^{512} + 1 = \prod_{i=0}^{255} (x^2 - \zeta^{2\text{bit_rev}_8(i)+1}),$$

being $\zeta = 62$ the choice in this case.

Therefore, the ring $R_q := \mathbb{Z}_q[X]/(X^n + 1)$ is isomorphic to T_q , where

$$T_q = \bigoplus_{i=0}^{n/2-1} \mathbb{Z}_q[X]/(X^2 - \zeta^{2\text{bit_rev}_r(i)+1}),$$

and the NTT representation $\hat{f} \in T_q$ of $f \in R_q$ is represented as:

$$f \mapsto \hat{f} = (f \bmod (X^2 - \zeta^{2\text{bit_rev}_r(i)+1}))_{i=0}^{n/2-1}.$$

Moreover, each transform block consists of two coefficients as:

$$f \bmod (X^2 - \zeta^{2\text{bit_rev}_r(i)+1}) = \hat{f}[2i] + \hat{f}[2i+1]X.$$

In this case, the output is split into consecutive pairs, and each pair represents a polynomial of degree < 2 in a quotient ring $\mathbb{Z}_q[X]/(X^2 - \Gamma)$. Hence, the natural multiplication subroutine is a degree-1 multiplication modulo $X^2 - \Gamma$. Algebraically,

$$(a_0 + a_1X)(b_0 + b_1X) = a_0b_0 + (a_0b_1 + a_1b_0)X + a_1b_1X^2,$$

and then X^2 is replaced by Γ . This multiplication is given by the algorithm `base_mul` (Alg. 6).

The concrete algorithms we use for the computation of these transformations and operations are provided in 2.2.2

Random oracles and XOFs. Throughout the specification, we assume the availability of cryptographic hash functions and extendable-output functions (XOFs) provided by ICCS. These are used for:

- Generating the public matrix **A** from a short seed.
- Sampling secrets and errors via rejection or bit-slicing.
- Deriving session keys from shared bit strings.

We denote generic hash/XOF functions by H, G , etc.; their precise instantiation follows SM3, HMAC-SM3 and KDF-SM3. These hash functions are used in the reference implementation just for correctness verification, with no security guarantee.

2.1.2 Noise Distributions

The security and correctness of DKE relies on carefully chosen distributions for secrets and error terms, mirroring ML-KEM. At a high level, these are *small* integer-valued distributions with support concentrated around 0 and short tails.

Centered binomial distribution. Let $\eta \in \mathbb{N}$ be a small integer parameter. The centered binomial distribution CBD_η over $\mathbb{Z}$ is defined as the distribution of the random variable

$$X = \sum_{i=1}^{\eta} (b_i - b'_i),$$

where $(b_i)_{i=1}^{\eta}$ and $(b'_i)_{i=1}^{\eta}$ are independent Bernoulli random variables with parameter $1/2$. Thus, $X \in \{-\eta, \dots, \eta\}$, $\mathbb{E}[X] = 0$, and $\text{Var}(X) = \eta$.

We denote by χ_η the discrete distribution on $\mathbb{Z}_q$ obtained by reducing samples from CBD_η modulo q .

Polynomial and vector-valued distributions. We lift χ_η to polynomials in R_q by sampling each coefficient independently:

$$a \leftarrow \chi_\eta^{R_q} \quad \text{means} \quad a(X) = \sum_{i=0}^{n-1} a_i X^i \quad \text{with} \quad a_i \leftarrow \chi_\eta \text{ i.i.d. for } 0 \leq i < n.$$

Similarly, for $k > 1$ we define $\chi_\eta^{R_q^k}$ as the product distribution on R_q^k obtained by sampling each component in R_q according to $\chi_\eta^{R_q}$.

Secret and error distributions. We consider the secret and error distribution, denoted χ_η , where η depends on the target security level (see Section 2.5). For a given parameter set, sampling

$$\mathbf{s}, \mathbf{e} \leftarrow \chi_\eta^{R_q^k}$$

means that each polynomial component and each of its coefficients are chosen independently according to the χ_η distribution. This distribution is chosen so that:

- Secrets and errors are sufficiently small to ensure correctness of the reconciliation mechanism (noise remains within decoding bounds with overwhelming probability).
- At the same time, the induced Module-LWE instances are believed to be hard for current classical and quantum attacks at the targeted security levels, as evidenced by ML-KEM's analysis [17], and is further reaffirmed by the analysis in Section 5.2.

2.1.3 Public Parameters

DKE public parameters. An instance of DKE is specified by a parameter set Π defined as follows:

$$\Pi = (R_q, q, n, k, \chi_\eta, \lambda, \lambda_K, \ell, d, G, H)$$

For the first two levels of security, this may be aligned with one of the ML-KEM parameter sets.

For a fixed choice of Π , the following quantities are public and shared by all users:

- The ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1)$$

with $q = 3329$ ($q = 7681$) and $n = 256$ ($n = 512$).

- The module rank $k \in \{2, 4\}$, determining the dimension of secrets and errors:

$$\mathbf{s}, \mathbf{e} \in R_q^k.$$

- The noise distribution χ_η for secrets and errors, as described in Section 2.1.2.
- A seed length λ (e.g. $\lambda = 256$ bits) for the public matrix seed $\rho \in \{0, 1\}^\lambda$.
- Cryptographic hash and XOF functions, modeled either as concrete primitives (e.g. SM3/KDF-SM3) or as random oracles:

– $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^*$, an XOF used to generate $\mathbf{A}$ from ρ .

– $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_K}$, a hash or KDF used to derive the final session key from the public key in DKEM.

- Signal parameter ℓ partitioning $\mathbb{Z}_q$ into 2^ℓ subsets for more fine-grained signals. This may be chosen to match the ML-KEM compression parameter d_v .
- A compression parameter d specifying how many bits are used to represent each coefficient of polynomials. This may be chosen to match the ML-KEM compression parameter d_u .

DKEM parameter sets. As mentioned above, the IND-CCA secure KEM described in 2.4, namely DKEM, is derived directly from the Key Exchange Protocol DKE. During this DKEM specification, the subjacent DKE protocol uses secret/error distribution χ_η equal to a centered binomial distribution; the output length of the shared secret and the length of seeds is always

equal to n (i.e., $n = \lambda = \lambda_K$), and the hash functions are provisionally provided by ICCS (namely, pseudoHash and pseudoXOF in NGCC). Hence, we only require

$$\pi = (q, n, k, \eta, \ell, d)$$

as a simplified, yet less flexible, collection of public parameters. In this sense, saying *DKEM is instantiated with a parameter set* $\pi = (q, n, k, \eta, \ell, d)$ means that the underlying DKE algorithms are defined with respect to

$$\Pi = (R_q, q, n, k, \chi_\eta, n, n, \ell, d, G, H),$$

where G and H are hash functions with at least n classical security bits (provisionally, pseudoHash and pseudoXOF provided by NGCC).

Concretely, we propose three instantiations for DKEM:

- **DKEM-128**: choosing the public parameter set $\pi_{128} = (3329, 256, 2, 3, 4, 10)$.
- **DKEM-256**: choosing the public parameter set $\pi_{256} = (3329, 256, 4, 2, 5, 11)$.
- **DKEM-512**: choosing the public parameter set $\pi_{512} = (7681, 512, 4, 3, 4, 11)$.

The numbers 128, 256 and 512 refer to the target classical security bits of each instantiation.

2.1.4 Matrix Generation and Sampling Algorithms

Following ML-KEM, the public matrix $\mathbf{A} \in R_q^{k \times k}$ is generated deterministically from a short seed ρ . This allows compact representation of the global public parameter and ensures that both parties obtain identical matrix entries without transmitting the entire matrix.

At the specification level, matrix generation (Alg. 1) is described through an auxiliary procedure SamplePoly, which is treated as a deterministic black-box sampler parameterized by the seed and the matrix coordinates. This keeps the specification aligned with the implemented behavior while avoiding commitment to a particular low-level XOF interface.

Algorithm 1 GenMatrix($\rho, transposed$)

Input: a seed $\rho \in \{0, 1\}^\lambda$ and a flag $transposed \in \{0, 1\}$

Output: $\mathbf{A}$ if $transposed = 0$, or $\mathbf{A}^\top$ otherwise

```
1: for  $i = 0$  to  $k - 1$  do
2:   for  $j = 0$  to  $k - 1$  do
3:     if  $transposed = 1$  then
4:        $\mathbf{A}_{i,j} \leftarrow \text{SamplePoly}(\rho, i, j)$ 
5:     else
6:        $\mathbf{A}_{i,j} \leftarrow \text{SamplePoly}(\rho, j, i)$ 
7:     end if
8:   end for
9: end for
```

We model $\text{SamplePoly}(\rho, x, y)$ as a deterministic, domain-separated procedure which, on input the public seed ρ and two coordinate values $i, j \in \{0, \dots, k - 1\}$, outputs a polynomial in R_q . For fixed inputs, the output is uniquely determined, and for uniformly random ρ the resulting coefficients are intended to be uniformly distributed over $\mathbb{Z}_q$. The concrete realization of this procedure, including how pseudorandom bytes are expanded and mapped to coefficients, is left to the implementation as long as the resulting behavior is consistent with this specification.

We also require an auxiliary sampling procedure:

Algorithm SampleBinomialPoly $_\eta(\mathbf{b})$: On input a bit string $\mathbf{b} \in \{0, 1\}^{2\eta n}$, compute a polynomial $\mathbf{s} \in R_q$ with coefficients in $[-\eta, \eta]$. If $\mathbf{b}$ is uniformly random, then the coefficients of $\mathbf{s}$ follow the centered binomial distribution χ_η . Such an algorithm is described in Algorithm 8.

For sampling an element $\mathbf{s} \leftarrow \chi_\eta^{R_q^k}$, we use SampleSecret $_\eta$, given in Algorithm 9.

2.2 Core Functions

The following core functions are used in the ephemeral DKE protocol (Section 2.3), which is evolved to a KEM protocol in Section 2.4.

2.2.1 Reconciliation Mechanism

The reconciliation mechanism of the original Ding Key Exchange [5, 6] is adapted here using more fine-grained signals; transmitting ℓ bits per coefficient (2^ℓ signal values) to reduce the failure rate is the general key-consensus technique of [18, 19], specialized in this proposal to the ML-KEM ring. The improved reconciliation mechanism consisting of the following two functions: Signal function (Sig) and Reconciliation function (Rec). In order to define them we partition the domain in the following way:

Partitioning the signal domain. Let ℓ be a positive integer such that $2^{\ell+1} \mid (q-1)$. Define $L = \frac{q-1}{2^\ell}$, then $\frac{L}{2}$ is an integer. The signal domain $\mathbb{Z}_q = \{-\frac{q-1}{2}, \dots, \frac{q-1}{2}\}$ is partitioned into 2^ℓ smaller subdomains:

$$\begin{aligned} I_0 &= \left\{ -\frac{L}{2}, \dots, \frac{L}{2} \right\}, \\ I_1 &= \left\{ \frac{L}{2} + 1, \dots, \frac{3L}{2} \right\}, \\ &\vdots \\ I_{2^{\ell-1}} &= \left\{ \frac{q-1-L}{2} + 1, \dots, \frac{q-1}{2} \right\} \cup \left\{ -\frac{q-1}{2}, \dots, -\frac{q-1-L}{2} - 1 \right\}, \\ &\vdots \\ I_{2^\ell-1} &= \left\{ -\frac{3L}{2}, \dots, -\frac{L}{2} - 1 \right\}. \end{aligned}$$

Signal function Sig. Let $\text{Sig} : \mathbb{Z}_q \times \{0, 1\} \rightarrow \{0, 1, \dots, 2^\ell - 1\}$. For an element $x \in \mathbb{Z}_q$, first sample b randomly from $\{0, 1\}$ with equal probability. If $x - b$ lies within the interval I_w , then define the signal of x as $\text{Sig}(x, b) = w$. The purpose of this random sampling is to eliminate statistical bias in the signal function. The signal function naturally extends to $\text{Sig} : R_q \times \{0, 1\}^n \rightarrow \{0, 1, \dots, 2^\ell - 1\}^n$ by applying it to each coefficient of a polynomial. For simplicity, when we denote $\text{Sig}(x)$, we implicitly assume a random b is already sampled. This function is specified in Algorithm 2.

Algorithm 2 $\text{Sig}(f, \mathbf{b})$ (signal function over R_q)

Input: $f \in R_q$, randomness $\mathbf{b} \in \{0, 1\}^n$;

Output: Signal value $\mathbf{w} \in \{0, 1, 2, \dots, 2^\ell - 1\}^n$.

- 1: Initialize $\mathbf{w} = \vec{0}$.
 - 2: **for** $i = 0, \dots, n-1$ **do**
 - 3: Compute the unique $w \in \{0, 1, 2, \dots, 2^\ell - 1\}$ such that $f[i] - \mathbf{b}[i] \pmod{q} \in I_w$.
 - 4: $\mathbf{w}[i] \leftarrow w$
 - 5: **end for**
 - 6: **return** $\mathbf{w}$.
-

Reconciliation function Rec. Let $\text{Rec} : \mathbb{Z}_q \times \{0, 1, \dots, 2^\ell - 1\} \rightarrow \{0, 1\}$. Given a signal w , an element $y \in \mathbb{Z}_q$ is mapped to 0 or 1 according to the following rule: if the signal is 0, indicating y lies in the interval I_0 , then y modulo 2 is taken directly, i.e., 0 if y is even, and 1 if odd. If the signal is $w > 0$, meaning in the interval $I_w \neq I_0$, then y is first reduced by wL , which effectively rotates the interval I_w back to I_0 , and is then taken modulo 2. Formally, this is expressed as:

$$\text{Rec}(y, w) = y - wL \pmod{q} \pmod{2}.$$

The reconciliation function naturally extends to $\text{Rec} : R_q \times \{0, 1, \dots, 2^\ell - 1\}^n \rightarrow \{0, 1\}^n$ by applying it to each coefficient of a polynomial. This function is specified in Algorithm 3.

Algorithm 3 $\text{Rec}(f, \mathbf{w})$ (Extractor over R_q)

Input: $f \in R_q, \mathbf{w} \in \{0, 1, \dots, 2^\ell - 1\}^n$.

Output: $\mathbf{ss} \in \{0, 1\}^n$.

```

1: for  $i = 0, \dots, n - 1$  do
2:    $\mathbf{ss}[i] = f[i] - \mathbf{w}[i]L \pmod{q} \pmod{2}$ 
3: end for
4: return  $\mathbf{ss}$ .

```

The partitioned reconciliation function has the following property: If the difference between elements $x, y \in \mathbb{Z}_q$ is even and less than or equal to $(q/2)(1 - 1/2^\ell) - 2$, then based on the signal of y , the reconciliation function maps x and y to the same value. This fact, formalized in Lemma 3.1, allows both communicating parties to reconcile and obtain the same value; that is, based on one party's signal, two numbers with the same parity and close values are mapped to the same value. When $\ell = 1$, this is consistent with the original Ding Key Exchange [5, 6]. As ℓ increases, the allowable difference grows from approximately $q/4 - 2$ to close to $q/2 - 2$, meaning the signal and reconciliation functions can tolerate larger errors. The role of partitioning the signal interval is that for the signal $w = \text{Sig}(y)$, the range of $y - wL$ is compressed from the original $\{-q/4, \dots, q/4\}$ to $\{-L/2, \dots, L/2\}$. This enables the reconciliation function $\text{Rec}(x, w) = x - wL = (x - y) + (y - wL)$ to tolerate a larger difference between x and y . Consequently, the sampling range allowed during random sampling is expanded, which greatly reduces the failure probability. But this also comes at the cost that the signal leaks ℓ bits for each coefficient.

We prove that even after we reveal the signal w the extracted bit is still perfectly unbiased.

Lemma 2.1. Let $q > 4$ be an odd integer and ℓ be a positive integer such that $2^{\ell+1} \mid (q - 1)$. If x is chosen uniformly random from $\mathbb{Z}_q$, then the reconciliation function $\text{Rec}(x, w)$ defined as above is uniformly random conditioned on the signal $\text{Sig}(x) = w \in \{0, 1, \dots, 2^\ell - 1\}$.

Proof. See Appendix A. □

2.2.2 Number-Theoretic Transform (NTT)

This section provides the concrete algorithms we use to compute the operations described in subsection 2.1.1 related to the NTT. We aim to stay consistent with the formulation of [8, 7, 17].

Algorithm 4 $\text{ntt}(f)$

Input: $f \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order n

Output: $\hat{f} \in \mathbb{Z}_q^n$

```

1:  $\hat{f} \leftarrow f$ 
2:  $r \leftarrow \log_2(n/2)$ 
3:  $i \leftarrow 1$ 
4: for  $\text{len} = n/2, n/4, \dots, 4, 2$  do
5:   for  $\text{start} = 0, 2\text{len}, 4\text{len}, \dots, n - 2\text{len}$  do
6:      $\zeta \leftarrow \zeta^{\text{bit\_rev}_r(i)} \bmod q$ 
7:      $i \leftarrow i + 1$ 
8:     for  $j = \text{start}, \text{start} + 1, \dots, \text{start} + \text{len} - 1$  do
9:        $t \leftarrow \zeta \hat{f}[j + \text{len}] \bmod q$ 
10:       $\hat{f}[j + \text{len}] \leftarrow \hat{f}[j] - t \pmod{q}$ 
11:       $\hat{f}[j] \leftarrow \hat{f}[j] + t \pmod{q}$ 
12:    end for
13:  end for
14: end for
15: return  $\hat{f}$ 

```

Algorithm 5 $\text{inv_ntt}(\hat{f})$

Input: $\hat{f} \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order $2n/\iota$

Output: $f \in \mathbb{Z}_q^n$

```

1:  $f \leftarrow \hat{f}$ 
2:  $r \leftarrow \log_2(n/2)$ 
3:  $i \leftarrow n/\iota - 1$ 
4: for  $\text{len} = 2, 4, \dots, n/2$  do
5:   for  $\text{start} \leftarrow 0, 2\text{len}, 4\text{len}, \dots, n - 2\text{len}$  do
6:      $\zeta \leftarrow \zeta^{\text{bit\_rev}_r(i)} \bmod q$ 
7:      $i \leftarrow i - 1$ 
8:     for  $j \leftarrow \text{start}, \text{start} + 1, \dots, \text{start} + \text{len} - 1$  do
9:        $u \leftarrow f[j]$ 
10:       $v \leftarrow f[j + \text{len}]$ 
11:       $f[j] \leftarrow u + v \pmod{q}$ 
12:       $f[j + \text{len}] \leftarrow \zeta(v - u) \pmod{q}$ 
13:    end for
14:  end for
15: end for
16:  $c \leftarrow (n/2)^{-1} \pmod{q}$ 
17: return  $c \cdot f$ 

```

▷ component-wise multiplication modulo q

Algorithm 6 $\text{base_mul}(a, b, \Gamma)$

Input: $a = (a_0, a_1)$, $b = (b_0, b_1) \in \mathbb{Z}_q^2$ and $\Gamma \in \mathbb{Z}_q$

Output: $r = (r_0, r_1) = (a_0 + a_1X)(b_0 + b_1X) \bmod (X^2 - \Gamma)$

1: $r_0 \leftarrow a_0b_0 + \Gamma a_1b_1 \pmod{q}$

2: $r_1 \leftarrow a_0b_1 + a_1b_0 \pmod{q}$

3: **return** (r_0, r_1)

Algorithm 7 $\text{ntt_mul}(\hat{a}, \hat{b})$

Input: $\hat{a}, \hat{b} \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order n

Output: $\hat{c} \in \mathbb{Z}_q^n$ ▷ We also denote this output as $\hat{a} \circ \hat{b}$

1: $r \leftarrow \log_2(n/2)$

2: **for** $i \leftarrow 0, 1, \dots, n/2 - 1$ **do**

3: $\Gamma \leftarrow \zeta^{2^{\text{bit_rev}_r(i)+1}} \bmod q$

4: $(\hat{c}[2i], \hat{c}[2i+1]) \leftarrow \text{base_mul}((\hat{a}[2i], \hat{a}[2i+1]), (\hat{b}[2i], \hat{b}[2i+1]), \Gamma)$

5: **end for**

6: **return** $\hat{c}$

- Algorithm 4 and Algorithm 5 compute the forward and inverse transforms using the classical Cooley–Tukey and Gentleman–Sande butterflies [20, 21], with the bit-reversed twiddle-factor ordering of [22]. We follow the concrete formulation also adopted in [8].
- Algorithm 6 performs the degree-one multiplication in $\mathbb{Z}_q[X]/(X^2 - \Gamma)$, and Algorithm 7 assembles the pointwise product $\hat{a} \circ \hat{b}$ over the $n/2$ blocks. Stopping at quadratic factors rather than splitting $X^n + 1$ completely, which permits the smaller moduli we use, follows [16, 23].
- We write NTT and NTT^{-1} for the transforms of Algorithms 4 and 5 applied to each coordinate of a vector of polynomials, and $\circ$ for the pointwise product of Algorithm 7, extended coordinate-wise to vectors and matrices.

2.2.3 Compression and Decompression Functions

We define the compression function for $x \in \mathbb{Z}_q$ as:

$$\text{Compress}_d(x) = \left\lceil \frac{2^d}{q} \cdot x \right\rceil \bmod 2^d. \quad (1)$$

For $y \in \mathbb{Z}_{2^d}$, the decompression function is:

$$\text{Decompress}_d(y) = \left\lceil \frac{q}{2^d} \cdot y \right\rceil. \quad (2)$$

Both functions are extended to R_q and R_q^k coefficient-wise. In these cases they are equally denoted for notation simplicity.

Remark 2.2. Compressing and decompressing the public MLWE samples is not possible in the original Ding Key Exchange [5]. This is because successively compressing and decompressing a value in $\mathbb{Z}_q$ introduces a not necessarily even error: let $r(x) = x - \text{Decompress}_d(\text{Compress}_d(x))$. Supposing that d is sufficiently close to $\lceil \log_2 q \rceil$, $|r(x)|$ will be close to 0. However, it might be an odd number, so compressing must be avoided in [5]. In [6], submitters introduced the so-called Round, Recover and RemoveBias functions to overcome this difficulty. Our approach is slightly different: each time we want to communicate a coefficient $x \in \mathbb{Z}_q$ preserving parity, instead of directly compressing x , we compress $\frac{1}{2}x$ (where $\frac{1}{2}$ denotes the inverse of 2 mod q) to obtain x' , and the receiver recovers $2 \cdot x' \bmod q$. It is easy to check that both properties hold: the error introduced is both small and even as long as d is sufficiently close to $\lceil \log_2 q \rceil$.

2.3 Specification of DKE

In this subsection we give a specification of DKE (Algorithms 8, 9, 10, 11 and 12), a two-pass key exchange protocol between an initiator A and a responder B . At the end of the protocol, both parties compute a shared secret $ss \in \{0, 1\}^n$. We discuss the correctness and reconciliation failure probability for this protocol in 3.4.

Algorithm 8 SampleBinomialPoly $_{\eta}(\mathbf{b})$

Input: $\mathbf{b} \in \{0, 1\}^{2 \cdot \eta \cdot n}$.

Output: $s \in R_q$ with coefficients sampled from $\mathcal{B}_{\eta}(\mathbb{Z}_q)$, as long as $\mathbf{b}$ is uniformly random.

```

1: for  $i = 0, \dots, n - 1$  do
2:    $a \leftarrow \sum_{j=0}^{\eta-1} \mathbf{b}[j + 2 \cdot i \cdot \eta]$ 
3:    $b \leftarrow \sum_{j=0}^{\eta} \mathbf{b}[j + \eta + 2 \cdot i \cdot \eta]$ 
4:    $s[i] \leftarrow a - b$ 
5: end for
   return  $s$ .
```

Algorithm 9 SampleSecret $_{\eta}(r, \text{nonce})$

Input: $r \in \{0, 1\}^n$, $\text{nonce} \in \mathbb{N}$

Output: $\mathbf{s} \in R_q^k$ with coefficients sampled from $\mathcal{B}_{\eta}(\mathbb{Z}_q)$.

```

1: for  $j = 0, \dots, k - 1$  do
2:    $\mathbf{b} \leftarrow \text{XOF}((r \parallel \text{nonce}); 2 \cdot \eta \cdot n)$ 
3:    $\mathbf{s}[j] \leftarrow \text{SampleBinomialPoly}_{\eta}(\mathbf{b})$ 
4:    $++ \text{nonce}$ 
5: end for
   return  $\mathbf{s}, \text{nonce}$ 
```

2.4 Specification of CCA-Secure DKEM

A key encapsulation mechanism, called DKEM, is constructed directly from DKE with the final key derived directly from the shared secret. This is specified in Algorithms 13, 14, 15 and 16.

Algorithm 10 DKE.Initiate(*coins*)

Input: randomness *coins* $\in \{0, 1\}^n$.

Output: public key *pk*.

Output: private key *sk*.

- 1: $(r, \rho) \leftarrow \text{XOF}(\text{coins}; 2n)$ $\triangleright$ expand to two pseudorandom n -bit seeds
 - 2: $\bar{\mathbf{A}} \leftarrow \text{GenMatrix}(\rho)$ $\triangleright$ generate matrix $\mathbf{A} \in (\mathbb{Z}_q^n)^{k \times k}$
 - 3: $(\mathbf{s}_A, \text{nonce}) \leftarrow \text{SampleSecret}_\eta(r, 0)$ $\triangleright$ generate secret $\mathbf{s}_A \in (\mathbb{Z}_q^n)^k$
 - 4: $\mathbf{e}_A \leftarrow \text{SampleSecret}_\eta(r, \text{nonce})$ $\triangleright$ generate error $\mathbf{e}_A \in (\mathbb{Z}_q^n)^k$
 - 5: $\bar{\mathbf{s}}_A \leftarrow \text{NTT}(\mathbf{s}_A)$ $\triangleright$ run NTT k -times (once for each coordinate of $\mathbf{s}_A$)
 - 6: $\bar{\mathbf{e}}_A \leftarrow \text{NTT}(\mathbf{e}_A)$ $\triangleright$ run NTT k -times (once for each coordinate of $\mathbf{e}_A$)
 - 7: $\bar{\mathbf{u}}_A \leftarrow \bar{\mathbf{A}} \circ \bar{\mathbf{s}}_A + \bar{\mathbf{e}}_A$ $\triangleright$ noisy linear system in NTT domain
 - 8: $pk \leftarrow \text{ByteEncode}(\bar{\mathbf{u}}_A) \parallel \rho$
 - 9: $sk \leftarrow \text{ByteEncode}(\bar{\mathbf{s}}_A)$
 - 10: **return** pk, sk
-

Algorithm 11 DKE.Response(*pk*, *coins*)

Input: public key *pk*.

$\triangleright$ initiator's public key *pk*

Input: random *coins* $\in \{0, 1\}^{2n}$.

Output: response message *c*.

$\triangleright$ responder's compressed data with signal

Output: shared secret $ss \in \{0, 1\}^n$.

- 1: $(\bar{\mathbf{u}}_A, \rho) \leftarrow pk$
 - 2: $(\mathbf{r}, \mathbf{b}) \leftarrow \text{coins}$
 - 3: $\bar{\mathbf{A}}^\top \leftarrow \text{GenMatrix}(\rho, 1)$
 - 4: $(\mathbf{s}_B, \text{nonce}) \leftarrow \text{SampleSecret}_\eta(\mathbf{r}, 0)$
 - 5: $\mathbf{e}_B \leftarrow \text{SampleSecret}_\eta(\mathbf{r}, \text{nonce})$
 - 6: $\bar{\mathbf{s}}_B \leftarrow \text{NTT}(\mathbf{s}_B)$
 - 7: $\bar{\mathbf{u}}_B \leftarrow \bar{\mathbf{A}}^\top \circ \bar{\mathbf{s}}_B$
 - 8: $\mathbf{u}_B \leftarrow \text{NTT}^{-1}(\bar{\mathbf{u}}_B)$
 - 9: $\mathbf{u}_B \leftarrow \mathbf{u}_B + \mathbf{e}_B$
 - 10: $\mathbf{u}'_B \leftarrow \text{Compress}(\mathbf{u}_B, d_B)$ $\triangleright$ compress responder's data
 - 11: $\bar{k}_B \leftarrow \bar{\mathbf{u}}_A \circ \bar{\mathbf{s}}_B$
 - 12: $k_B \leftarrow \text{NTT}^{-1}(\bar{k}_B)$
 - 13: $\mathbf{r} \leftarrow \text{XOF}(\mathbf{r} \parallel \text{nonce}; 2 \cdot \eta \cdot n)$
 - 14: $e_B \leftarrow \text{SampleBinomialPoly}(\mathbf{r})$
 - 15: $k_B \leftarrow k_B + e_B$
 - 16: $k_B \leftarrow 2 \cdot k_B$
 - 17: $\mathbf{w} \leftarrow \text{Sig}(k_B, \mathbf{b})$ $\triangleright$ generate the signal
 - 18: $c \leftarrow (\mathbf{u}'_B \parallel \mathbf{w})$ $\triangleright$ generate response message from responder's data and signal
 - 19: $ss \leftarrow \text{Rec}(k_B, \mathbf{w})$ $\triangleright$ derive the shared secret by applying error reconciliation
 - 20: **return** (c, ss)
-

2.5 Parameter Sets

We summarize the parameter sets of DKEM in Table 1.

Algorithm 12 DKE.DeriveSecret(c, sk)

Input: message c . ▷ responder's compressed data with signal
Input: private key $sk = \bar{s}_A$.
Output: shared secret $ss \in \{0, 1\}^n$.
1: $(\mathbf{u}'_B, \mathbf{w}) \leftarrow c$
2: $\mathbf{u}''_B \leftarrow \text{Decompress}(\mathbf{u}'_B, d_B)$
3: $\bar{\mathbf{u}}''_B \leftarrow NTT(\mathbf{u}''_B)$
4: $\bar{k}_A \leftarrow \bar{\mathbf{u}}''_B \circ \bar{s}_A$
5: $k_A \leftarrow 2 \cdot NTT^{-1}(\bar{k}_A)$
6: $ss \leftarrow \text{Rec}(k_A, \mathbf{w})$ ▷ derive the shared secret by applying error reconciliation
7: **return** ss

Algorithm 13 DKEM.KeyGen()

Output: encapsulation key ek .
Output: decapsulation key dk .
1: $coins \leftarrow \{0, 1\}^n$
2: $pk, sk \leftarrow \text{DKE.Initiate}(coins)$
3: $rej \leftarrow \{0, 1\}^n$ ▷ randomness for implicit rejection in case of failure
4: $dk = (sk || pk || rej)$
5: **return** $(ek = pk, dk)$

Algorithm 14 DKEM.Internal($ek, coins$)

Input: encapsulation key $ek = (\dots || \rho)$.
Input: randomness $coins \in \{0, 1\}^n$.
Output: ciphertext ct .
Output: shared key $K \in \{0, 1\}^n$.
1: $\rho \leftarrow ek$ ▷ ρ comes from the last n bits of ek
2: $r \leftarrow \text{XOF}((coins || \rho); 2n)$
3: $c_1, ss \leftarrow \text{DKE.Response}(ek, r)$
4: $c_2 \leftarrow coins \oplus ss$ ▷ mask the randomness by the shared secret
5: $ct = (c_1 || c_2)$
6: $K \leftarrow \text{KDF}(ss; n)$ ▷ generate the shared key from shared secret
7: **return** (ct, K)

Algorithm 15 DKEM.Encaps(pk)

Input: encapsulation key ek .
Output: ciphertext ct .
Output: shared key $K \in \{0, 1\}^n$.
1: $coins \leftarrow \{0, 1\}^n$
2: $ct, K \leftarrow \text{DKEM.Internal}(ek, coins)$. ▷ encrypt the random coins and derive the shared key
3: **return** (ct, K)

Algorithm 16 DKEM.Decaps(dk, ct)

Input: decapsulation key dk .

Input: ciphertext $ct = (c_1 || c_2)$.

Output: shared key $K \in \{0, 1\}^n$.

```
1:  $(sk, ek, rej) \leftarrow dk$ 
2:  $(c_1, c_2) \leftarrow ct$ 
3:  $ss \leftarrow \text{DKE.DeriveSecret}(c_1, sk)$   $\triangleright$  derive the shared secret of key exchange
4:  $coins \leftarrow c_2 \oplus ss$   $\triangleright$  unmask to recover the randomness
5:  $(ct', K) \leftarrow \text{DKEM.Internal}(ek, coins)$   $\triangleright$  re-construct using the derived randomness
6:  $\tilde{K} \leftarrow \text{KDF}(rej || c_2; n)$ 
7: if  $ct \neq ct'$  then
8:   return  $\tilde{K}$   $\triangleright$  if ciphertexts do not match, “implicitly reject”
9: end if
10: return  $K$ 
```

Name	n	q	k	η	ℓ	d	Failure prob.	NGCC level	NIST equivalent
DKEM-128	256	3329	2	3	4	10	$2^{-132.7}$	1	ML-KEM-512
DKEM-256	256	3329	4	2	5	11	$2^{-181.2}$	2	ML-KEM-1024
DKEM-512	512	7681	4	3	4	11	$2^{-167.0}$	3	-

Table 1: Summary of parameter sets for DKEM. The failure probability column has been calculated as described in Section 3.4, and using the scripts in [24].

3 Design Rationale

3.1 Design goals and the ICCS context

The design of DKEM is completely shaped by the NGCC submission requirements of the PQC standardization call, which explicitly requests materially distinct constructions from already standardized ones (Requirement 1.5 in [25]). At the same time, any post-quantum KEM intended for real-world deployment must be trusted, and cryptographic trust is earned through sustained public analysis of concrete constructions, not through novelty alone. DKEM therefore addresses both pressures at the same time: while the underlying algebraic structure on which it operates as well as some of the parameters are conservatively inherited from the long studied ML-KEM (standardized by NIST in August 2024 as FIPS203 [8]), the cryptographic construction of DKEM — how a shared secret is derived from the underlying hard problem, how it is bound to the ciphertext and how CCA security is achieved — are new.

The design of DKEM is thus built around two explicit pillars. The first is distinct lineage. Concretely, none of the reconciliation-based CPA core (DKE) based in [6], in which message encryption is replaced; the modulo 2 robust extractor, in which the shared secret is extracted as one bit per ring coefficient rather than by recovering a fixed-length plaintext; the random-

ized doubling, which removes the bias introduced by odd moduli in the signal function; or the tag-based Fujisaki-Okamoto transform appear in ML-KEM or in any previously standardized module-lattice KEM. Moreover, DKEM also adds an additional parameter set to achieve the target 512 classical security-bits, a non-existent level in ML-KEM. The second fundamental pillar is conservative trust. The underlying mathematical structure is directly inherited from ML-KEM. Concretely, the first two parameter sets, the NTT arithmetic and the CBD sampling are maintained because they have been more thoroughly analyzed than those of any alternative, across three rounds of NIST evaluation and several years of post-standardization cryptanalytic research. This is why reuse is an active choice to let the accumulated confidence of the international cryptographic community in these components serve as a second layer of assurance beneath the novel construction. The principal design decisions of DKEM correspond to the NGCC Evaluation Criteria [25], with full corroboration provided in the Section 6.

3.2 The mathematical hard problem

The mathematical hard problem underlying DKEM is the Module Learning with Errors (MLWE), following the same rationale as ML-KEM. We explain this problem and how it relates to the security of DKE and DKEM in subsection 5.1.2. Previous lattice-based proposals have either relied on the highly structured Ring-LWE problem [26] or on standard LWE [27]. Ring-LWE-based schemes offer excellent efficiency in terms of speed and key and ciphertext sizes, but the additional algebraic structure raises concerns about potentially more efficient attacks and allows only coarse scaling of the security-efficiency tradeoff. Standard LWE avoids this structure entirely and scales more flexibly, but at the cost of significantly larger keys and ciphertexts and considerably worse performance. Module-LWE offers a middle ground between these two extremes: by working over a module of small rank k over a fixed polynomial ring, the scheme retains performance close to Ring-LWE while allowing fine-grained scaling of the security level simply by adjusting k , without modifying the underlying ring or its associated arithmetic. Furthermore, the module structure provides somewhat reduced algebraic regularity compared to Ring-LWE, which is a conservative choice regarding algebraic attacks against structured lattice problems [28, 29].

In this context, ML-KEM uses MLWE as the foundation for a public key encryption scheme in the LPR framework: a message m is encrypted and decrypted to use as shared secret. On the other hand, DKEM takes a different approach: MLWE is used as the foundation for a key exchange protocol based on Ding’s reconciliation mechanism [5, 6], where both parties hold noisy approximations of the same lattice product and use the improved signal function to agree on a shared secret without decryption. Thus, the MLWE assumption is the common root in both cases, while the primitive built above differs in each case.

3.2.1 The CPA reconciliation core (DKE)

The starting point of the CPA reconciliation core (DKE) is the fact that, as observed in [5], if two parties hold a MLWE sample derived from a shared public matrix, they can independently compute quantities that are close to the same lattice product, and with a reconciliation mechanism they can extract a common bit string from these values. For this aim the signal function is used, where the interval $[-(q-1)/2, (q-1)/2]$ is divided into 2^ℓ subintervals, and ℓ bits per coefficient are sent to express in which interval the value is. This step is needed to conserve the parity of each value. In fact, when reconciling a coefficient from the signal, the value is effectively rotated to the center of the interval before applying modulo 2 reduction. Without this step, all coefficients that have an extra modulo q computation change parity and thus do not yield the same value.

To avoid any statistical bias from the fact that odd moduli divide the interval in uneven-length subintervals, a randomized doubling step is inserted: instead of directly calculating the signal of a coefficient, the signal of that coefficient minus a random $b \in \{0, 1\}$ is calculated. Lemma 2.1 proves that all bias is removed with this method.

This reconciliation mechanism completely differs from ML-KEM or any other LPR cryptosystem, since no message is needed to encrypt and no decryption is performed.

3.2.2 CCA transform with tag-based Fujisaki-Okamoto

DKEM is a CCA secure scheme constructed by applying a FO-like transform to DKE. Unlike ML-KEM, where the FO transform consists in reencrypting the message, in DKEM's FO-like transform the randomness is masked with the shared secret during the encapsulation process, and is then unmasked in the decapsulation. Then, the ciphertext generation part of the encapsulation process is run again, and if the new ciphertext does not coincide with the original one, implicit rejection is done.

3.3 Criteria for Parameter Selection

The parameter sets for DKEM are chosen to provide an optimal balance between communication overhead, computational efficiency, and the rigorous security requirements defined by the NGCC. A fundamental design constraint is the maintenance of a negligible failure probability while ensuring resistance against both classical and quantum cryptanalysis. See Appendix B for an extensive comparison of all these variables for different choices of parameter sets.

3.3.1 Alignment with ML-KEM and Architectural Simplification

Our 128-bit and 256-bit security parameter sets align with the algebraic foundations of ML-KEM-512 (NIST security Level 1) and ML-KEM-1024 (NIST security Level 5). By aligning

the base ring $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ and modulus $q = 3329$ with global standards, DKEM directly leverages the extensive community know-how in optimized Number Theoretic Transform (NTT) arithmetic and constant-time random sampling (e.g. CBD).

A primary architectural refinement in DKEM with respect to ML-KEM is the adoption of a unified noise parameter η , used consistently across all secret and error vectors, replacing the dual-parameter approach (η_1, η_2) of ML-KEM. In ML-KEM, the distinction between η_1 and η_2 was introduced specifically for the ML-KEM-512 parameter set, where $\eta_1 = 3$ and $\eta_2 = 2$. The rationale is to balance the effect of the deterministic noise added in the compression step. For ML-KEM-768 and ML-KEM-1024, this distinction is unnecessary and both parameters are already set equal ($\eta_1 = \eta_2 = 2$), confirming that the dual-parameter design is a targeted optimization rather than a fundamental architectural requirement.

This design choice of a unified noise parameter for DKEM is motivated by the following:

- **Specification and implementation simplicity and efficiency** A single noise parameter eliminates an entire axis of parameter complexity from the specification. The sampling pipeline is unified: a single instantiation of CBD_η covers all noise generation, reducing code footprint, simplifying formal verification, and decreasing the risk of implementation errors arising from parameter confusion. This is particularly relevant for constrained environments such as embedded or hardware implementations, where reduced logic complexity has direct cost and security benefits.
- **Enhanced security margin:** For each security level, DKEM adopts the larger of the two original ML-KEM noise parameters. Concretely, $\eta = 3$ for the 128-bit security level and $\eta = 2$ for the 256-bit security level. This ensures that the noise variance across all components is at least as large in the corresponding ML-KEM parameter set, preserving or exceeding the intended security margin against lattice reduction algorithms such as BKZ [30] and its variants [31].
- **Negligible impact on failure probability.** The reconciliation failure probability of the scheme is computation is fully explained in Section 3.4. For the 256-bit security level, the unified $\eta = 2$ is identical to ML-KEM-1024, but for the 128-bit security level, adopting $\eta = 3$ increases the noise variance of the error terms $\mathbf{e}_B$ and e_B relative to ML-KEM-512, where $\eta_2 = 2$. This results in a modest increase in δ for this parameter set only. However, the resulting failure probability remains cryptographically negligible and well adhering to the strict reliability constraints of the NGCC.

Regarding the number of signal bits, as evidenced by the results in Table 8 (See Appendix B), utilizing $\ell = 4$ signal bits per coefficient for the 256-bit security level represents a viable and efficient configuration. This parameterization ensures that the decryption failure probability remains cryptographically negligible, adhering to the strict reliability constraints of the NGCC. Given the potential trade-offs between bandwidth efficiency and failure bounds, the

implications of adopting $\ell = 4$ in this context warrant further detailed discussion. We have provisionally implemented the more conservative choice of $\ell = 5$, aligning the choice with the corresponding parameters in ML-KEM-1024.

3.3.2 High-Security Parameter Set (512-bit)

To satisfy requirements for 512-bit classical security, a third parameter set is introduced. This set maintains the unified design philosophy while scaling the underlying structure to meet advanced security targets:

- **Base Ring and Optimized Arithmetic:** We define $R_q = \mathbb{Z}_q[X]/(X^{512} + 1)$ with $q = 7681$. The degree $n = 512$ is selected to ensure the shared secret length satisfies the target security level, since it is required that the length of the shared secret key is no less than the corresponding classical security strength level bits (Section 2.1 of the submission requirements in [25]). The modulus q is chosen such that $n \mid (q - 1)$. As it was shown in [16], the high-performance NTT-based arithmetic is fully achievable without the stricter $2n \mid (q - 1)$ constraint, allowing for a more efficient modulus-to-security ratio. Note that $q = 7681$ already appears in the Kyber round 1 literature [7], so it already has cryptanalytic exposure.
- **Module Rank and Noise:** A rank of $k = 4$ is utilized as the minimum necessary to achieve 512-bit security given the chosen n and q . In accordance with our simplified architecture, we utilize a single noise parameter $\eta = 3$, which is chosen for being the minimum necessary to achieve the requested classical security bits.
- **Bandwidth Optimization:** The compression parameters and the number of signal bits-per-coefficient ℓ are meticulously tuned. This calibration minimizes the total communication cost (bandwidth) while ensuring that the probability of reconciliation failure remains cryptographically negligible.

See Table 9 in Appendix B for an example of different choices for these parameters.

3.4 Failure probability analysis.

In this subsection, we analyze the correctness of DKEM and its failure probability. This analysis relies on Lemma 3.1 and Proposition 3.2, addressing the error tolerance of the subjacent DKE protocol. The correctness of DKEM immediately follows from the correctness of the core protocol DKE, and hence the failure probability of DKEM is upper-bounded by the failure probability of DKE.

The failure mechanism in DKEM differs structurally from the decryption failure model of ML-KEM, and the analysis must be carried out independently even for the parameter sets that

are shared. In ML-KEM, failure occurs when a rounding error in recovering the transported message m exceeds the decompression tolerance at the most significant bit. In DKEM there is no transported message: failure is a reconciliation failure, meaning that the noise term is larger than the signal tolerance, that is, the signal w computed by the responder from k_B cannot guide the initiator's reconciliation of k_A to the same value. This error term has three structurally distinct sources. Firstly, the cross-products $\mathbf{e}_A \cdot \mathbf{s}_B$ and $\mathbf{e}_B \cdot \mathbf{s}_A$ arise from the approximate nature of the LWE key exchange: both parties compute approximations to the same value, and the difference is exactly the product of one party's error with the other party's secret. Secondly, the compression residual $\mathbf{r}_B := \text{Decompress}(\text{Compress}(\mathbf{u}_B, d_B), d_B) - \mathbf{u}_B$ arises from transmitting $\mathbf{u}_B$ at d_B bits per coefficient rather than in full; this is the DKEM analogue of the d_u -compression noise in ML-KEM, but it enters the error through a different algebraic path. Finally, the responder's extra error e_B , added before the signal computation contributes an additional error term.

Lemma 3.1. Let $k_A, k_B \in R_q$, let $\mathbf{w} = \text{Sig}(k_B)$, and let ℓ be the number of signal bits per coefficient. Denote $E = k_A - k_B$. If the inequality

$$\|E\|_\infty \leq \text{tol}_\ell := \frac{q-1}{2} \left(\frac{2^\ell - 1}{2^\ell} \right) - 1$$

holds and E consists only on even coefficients, then $\text{Rec}(k_A, \mathbf{w}) = \text{Rec}(k_B, \mathbf{w})$. Sig and Rec denote the signal function defined in Algorithm 2, and the reconciliation function defined in Algorithm 3, respectively.

Proof. See Appendix A. □

Proposition 3.2. The protocol described in 2.4 outputs equal shared secrets $ss_A = ss_B$ if, for the secret and error vectors sampled during a run of such protocol — namely $\mathbf{s}_A, \mathbf{e}_A, \mathbf{s}_B, \mathbf{e}_B, e_B$ —, the following inequation holds:

$$\|\mathbf{e}_A \cdot \mathbf{s}_B - (\mathbf{e}_B + \mathbf{r}_B) \cdot \mathbf{s}_A + e_B\|_\infty \leq \frac{\text{tol}_\ell}{2},$$

where $\mathbf{r}_B := \mathbf{u}_B'' - \mathbf{u}_B$, and tol_ℓ is defined as in Lemma 3.1.

Proof. See Appendix A. □

Failure probability estimation. The strategy to estimate the failure probability of the Protocol described in 2.4 follows the approach used in [7].

Denote $E := \mathbf{e}_A \cdot \mathbf{s}_B - (\mathbf{e}_B + \mathbf{r}_B) \cdot \mathbf{s}_A + e_B$. Assuming $\mathbf{s}_A, \mathbf{e}_A, \mathbf{s}_B, \mathbf{e}_B \leftarrow \chi_\eta^{R_q^k}$, $e_B \leftarrow \chi_\eta^{R_q}$; since $\mathbf{r}_B$ only depends on $\mathbf{s}_B, \mathbf{e}_B$ through the pseudo-random value $\mathbf{u}_B$, we can see $\mathbf{r}_B$ as sampled from χ_η (independently from $\mathbf{s}_B, \mathbf{e}_B, e_B'$) where χ_η is the distribution obtained from sampling $\mathbf{u}_B \leftarrow R_q^k$ uniformly at random and outputting $\mathbf{u}_B - \text{Decompress}(\text{Compress}(\mathbf{u}_B, d), d)$. Under this assumption, we can explicitly compute the distribution of each coefficient of E when running the protocol, and from that computation, estimate the probability $\delta = \delta(\Pi)$ of the event

$\|E\|_\infty \leq \frac{\text{tol}}{2}$. We provide a complete tool for computing this estimation for user-defined parameter sets [24]. Finally, the probability of this event is related to the failure probability of DKEM via Proposition 3.2.

As seen in Table 1, the failure probability is $2^{-132.7}$, $2^{-181.2}$ and $2^{-167.0}$ for DKEM-128, DKEM-256 and DKEM-512 respectively. All these values are cryptographically negligible. For context, ML-KEM-512 and ML-KEM-1024 achieve failure probabilities of 2^{-139} and 2^{-174} respectively. The failure probability of DKEM-128 is slightly higher than the one of ML-KEM-512, as expected from adopting a unified $\eta = 3$ parameter, increasing the noise variance for all error vectors. DKEM-256 achieves slightly lower failure probability than the one of ML-KEM-1024 despite sharing the same ring and noise distributions, reflecting differences in how the compression noise and signal quantization contribute to the reconciliation error.

Security impact for non-zero failure probability. At a concrete level, the magnitude of the failure probability across all three parameter sets makes any attack requiring failure observations entirely impractical. The NGCC security evaluation model (Criteria 1.2 in [25]) bounds the adversary’s query budget at 2^{80} chosen-ciphertext decapsulation queries. Under this bound, the expected number of genuine reconciliation failures the adversary can observe is at most $2^{-52.7}$, $2^{-101.2}$ and $2^{-87.0}$ for DKEM-128, DKEM-256 and DKEM-512 respectively. In each case the expected number of observable failure events is negligibly small even granting the adversary the maximum permitted query budget, confirming that the failure probability introduces no exploitable weakness under the NGCC threat model.

3.5 Other considerations

Key reuse protection. The underlying CPA-secure DKE is a complete key-exchange protocol in its own right. By itself, however, it is suitable only for ephemeral use: under static key reuse it becomes vulnerable to key-mismatch attacks [32, 33, 34]. Reusing a key safely requires CCA security, which is precisely what DKEM provides.

Public key binding to the randomness. In Line 2 of Alg. 14, the last n bits of the encapsulation key are bound to the encapsulation randomness. Since distinct users’ keys yield distinct values of ρ , this binding produces distinct randomness for each target and thereby thwarts multitarget attacks (see Section 5.2.5).

Contributory nature of key derivation. DKEM — and even the underlying DKE — is contributory by construction. It is built on a reconciliation mechanism in which each party derives the shared secret from the other party’s public key using the signal, so both parties contribute to the result. This contrasts with ML-KEM, which attains contributiveness by hashing the public key together with the message during encapsulation. DKEM needs no such step to be contrib-

utary; the only value folded into the randomness is the last n bits of the encapsulation key, and that is done solely to prevent multitarget attacks.

We deliberately avoid hashing the full ciphertext and public key (or any digest thereof) into the final shared secret. This keeps the design modular: we delegate the choice of whether to bind pk and/or ct to the higher-level protocols instantiated with DKE or DKEM, each according to its own security requirements.

This choice also yields efficiency gains over ML-KEM in both encapsulation and decapsulation. As noted above, hashing the final bits of the encapsulation key into the encapsulator’s randomness already suffices to prevent multitarget attacks, and hashing the final bits of the ciphertext into the rejection value already suffices to keep the decapsulator from exposing the same rejection value across ciphertexts with distinct tags.

4 Performance Evaluation

Implementation scope and provenance. DKEM is provided in four implementations: a portable C implementation (Table 2), an AVX2-optimized implementation for x86-64 processors (Table 3), an ARMv8-A NEON implementation, and an optimized implementation for ARM Cortex-M4 targets (Table 5). All four implementations expose the same public KEM interface for key generation, encapsulation, and decapsulation. The portable implementation adapts low-level polynomial arithmetic, NTT, modular-reduction, sampling, packing, serialization, and constant-time programming techniques from the CRYSTALS-Kyber reference implementation and the PQClean ML-KEM code base [35, 12, 8]. The DKEM-specific CPA construction, Ding-style reconciliation mechanism, and tag-based CCA transform are implemented from this specification. The reference symmetric layer uses the ICCS SM3-based pseudoXOF, pseudoHash, and deterministic random-number generator. A file-level record of reused and adapted material is included in NOTICE.md in the implementation package.

Parameter set	Operation	Avg. time (μ s)	Avg. cycles	Throughput (ops/s)	Static image (B)	Peak RSS (KB)
DKEM-128	KeyGen	66.72	159 721	14 988.010	35 757	1 876
DKEM-128	Encaps	80.21	192 088	12 467.273	35 757	1 876
DKEM-128	Decaps	94.73	226 929	10 556.318	35 757	1 876
DKEM-256	KeyGen	212.80	509 765	4 699.248	36 957	1 876
DKEM-256	Encaps	211.79	507 604	4 721.658	36 957	1 876
DKEM-256	Decaps	229.81	550 777	4 351.421	36 957	1 876
DKEM-512	KeyGen	648.52	1 555 048	1 541.972	38 461	1 876
DKEM-512	Encaps	689.76	1 654 277	1 449.780	38 461	1 876
DKEM-512	Decaps	741.85	1 778 708	1 347.981	38 461	1 876

Parameter set	Public key (B)	Secret key (B)	Ciphertext (B)	Shared secret (B)
DKEM-128	800	1 600	800	32
DKEM-256	1 568	3 136	1 600	32
DKEM-512	3 392	6 784	3 136	64

Table 2: Portable C implementation. Performance values are arithmetic means; transmission and storage sizes are reported in bytes.

Benchmarking configuration. Measurements were collected on an Intel Core i9-10885H at 2.40 GHz with 64 GiB of RAM, running Ubuntu 24.04 under Microsoft WSL2 (kernel 6.x). The implementation was compiled with GCC 13.3.0 using `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-O2`, `-fcommon`, `-DDKE_FORCE_SCALAR`, `-DDKE_MODE=<128|256|512>`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`. Each entry is the arithmetic mean of 300 measured operations on a single core. Execution time was measured with `clock_gettime(CLOCK_MONOTONIC)`, and CPU cycles were measured with RDTSC; throughput is derived from the average execution time. Static image size is `text+data+bss` as reported by GNU `size`, and peak RSS is reported by `getrusage(RUSAGE_SELF)`. All KAT and deterministic shared-secret tests passed.

AVX2 implementation. The AVX2 implementation retains the high-level DKEM logic and public KEM API, while replacing the performance-critical polynomial layer with vectorized arithmetic. Under GCC, the optimized build links the hand-written GNU assembly routines `fq.S`, `ntt.S`, `invntt.S`, `basemul.S`, and `shuffle.S`. These routines accelerate modular arithmetic, the forward and inverse NTTs, base multiplication, matrix–vector accumulation, and coefficient rearrangement. DKEM-128 and DKEM-256 follow the optimization structure used by the corresponding Kyber/ML-KEM parameter sets, whereas DKEM-512 uses constants and transforms specialized for $n = 512$ and $q = 7681$ [35, 12].

Parameter set	Operation	Avg. time (μ s)	Avg. cycles	Throughput (ops/s)	Static image (B)	Peak RSS (KB)
DKEM-128	KeyGen	40.21	105 171	24 869.435	130 849	1 876
DKEM-128	Encaps	43.31	113 311	23 089.356	130 849	1 876
DKEM-128	Decaps	45.00	117 731	22 222.222	130 849	1 876
DKEM-256	KeyGen	131.72	329 137	7 591.862	133 043	1 876
DKEM-256	Encaps	132.42	330 887	7 551.729	133 043	1 876
DKEM-256	Decaps	135.09	337 571	7 402.472	133 043	1 876
DKEM-512	KeyGen	549.65	1 317 471	1 819.340	123 167	1 876
DKEM-512	Encaps	565.40	1 355 246	1 768.659	123 167	1 876
DKEM-512	Decaps	569.88	1 365 954	1 754.755	123 167	1 876

Table 3: AVX2 Performance Optimization Version. Performance values are arithmetic means.

Benchmarking configuration. The processor, operating system, timing method, sample count, and memory definitions are identical to those reported for Table 2. The implementation was compiled with GCC 13.3.0 using `-O3`, `-march=x86-64`, `-mavx2`, `-mtune=native`, `-flto`, `-fomit-frame-pointer`, `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-fcommon`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`, together with the linked GNU AVX2 assembly sources. All KAT and deterministic shared-secret tests passed.

ARMv8-A implementation. The ARMv8-A implementation retains the high-level DKEM logic and public KEM API, while replacing the performance-critical polynomial layer with vectorized arithmetic using ARM NEON. It provides hand-optimized routines for NTT/INTT, base multiplication, and polynomial operations. The reference SM3/pseudoXOF is used to generate pseudorandom number in DKEM. Note that this arm64 platform has no stable user-space cycle counter (PMU restricted), so average cycles are not reported; average wall-clock microseconds (μ s) are authoritative and used in Table 4.

Parameter set	Operation	Avg. time (μ s)	Throughput (ops/s)	Peak RSS (KB)
DKEM-128	KeyGen	39.05	25 608.19	1 776
	Encaps	42.14	23 730.42	1 776
	Decaps	43.32	23 084.03	1 776
DKEM-256	KeyGen	134.25	7 448.79	1 776
	Encaps	138.08	7 242.18	1 776
	Decaps	139.44	7 171.54	1 776
DKEM-512	KeyGen	501.63	1 993.50	1 776
	Encaps	512.78	1 950.15	1 776
	Decaps	517.89	1 930.91	1 776

Table 4: Average time, throughput and peak memory of the optimized ARMv8-A implementation. Throughput is derived from the average time.

Benchmarking configuration. Measurements were collected on an ARMv8.2-A architecture, single-core measurement. The implementation was built with the GNU GCC 13 using `-O3`, `-march=armv8.2-a`, `-flto`, `-fomit-frame-pointer`, `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-fcommon`, `-DDKE_USE_AARCH64`, `-DDKE_USE_AARCH64_NATIVE`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`. The average time entries are arithmetic means of 300 measurements. Each iteration is timed with `clock_gettime(CLOCK_MONOTONIC)`. Peak RSS is process peak resident memory reported by `getrusage(RUSAGE_SELF)`.

ARM Cortex-M4 implementation. The additional Cortex-M4 implementation provides optimized Plantard arithmetic, matrix accumulation, NTT/INTT, base multiplication, polynomial operations, and an assembly-optimized SM3 backend. The submitted self-test harness executes the implementation under QEMU and reports operation-level SysTick cycle counts.

Parameter set	Operation	Minimum cycles
DKEM-128	KeyGen	44 789
	Encaps	54 230
	Decaps	65 453
DKEM-256	KeyGen	161 028
	Encaps	160 087
	Decaps	176 719
DKEM-512	KeyGen	590 469
	Encaps	628 085
	Decaps	707 952

Table 5: Cortex-M4 operation cycle counts from the QEMU self-test harness. These results are additional embedded-platform evidence.

Benchmarking configuration. The Cortex-M4 implementation was built with arm-none-eabi-gcc 14.2.1 using `-mcpu=cortex-m4`, `-mthumb`, `-mfloat-abi=hard`, `-mfpv4-sp-d16`, `-O2`, `-ffunction-sections`, `-fdata-sections`, `-std=gnu99`, `-nostdlib`, and `-Wl,--gc-sections`. It was executed under QEMU on the netduinoplus2 Cortex-M4 target. Cycle counts were obtained from the 24-bit SysTick down-counter and are the minimum values observed over five self-test iterations for each operation. The implementation uses deterministic inputs, Plantard arithmetic, matrix accumulation, and the SM3 backend. The Cortex-M4 results are outside the ARMv8.2-A guideline scope and are provided as additional evidence only.

Comparison with ML-KEM. A separate controlled experiment compared the portable DKEM-128 and DKEM-256 implementations with the clean PQClean implementations of ML-KEM-512 and ML-KEM-1024, respectively, using matched compiler settings, platform, CPU affinity, timing method, and sampling protocol. DKEM required higher cycle counts than the corresponding ML-KEM implementations. Replacing only the SM3/pseudoXOF wrappers with SHA3/SHAKE wrappers, while leaving the DKEM arithmetic and protocol logic unchanged, substantially reduced the measured DKEM cost. This identifies the symmetric layer as a major contributor to DKEM’s software cost, while not attributing the complete difference to hashing because the schemes also differ in reconciliation and CCA-transform logic. No direct ML-KEM comparison is reported for DKEM-512 because ML-KEM has no standardized parameter set with matching dimensions.

5 Security Statements and Analyses

5.1 Theoretical security

This section provides a game-based security analysis of DKEM. The proof first establishes the pseudorandomness of the underlying DKE shared secret, and then uses this property to prove the IND-CCA security of the DKEM encapsulation transform.

5.1.1 Security model and proof outline

All adversaries considered in this section are probabilistic polynomial-time algorithms. The seed-expansion XOF and the KDF calls used in DKEM are modeled as domain-separated random oracles¹. The analysis is organized in two layers. First, we establish that the DKE shared secret is pseudorandom given the public key and the responder transcript, under the decisional Module-LWE assumption and the statistical unbiasedness of the reconciliation procedure on uniform inputs. Second, we prove the IND-CCA security of the DKEM encapsulation transform in the Random-Oracle Model (ROM). The CCA proof relies on the deterministic seed-recomputation check in decapsulation, the implicit-rejection branch, and the reconciliation-failure bound supplied by the correctness analysis.

5.1.2 The Module-LWE Assumption

The security of DKEM is based on the hardness of the Module-LWE problem over the ring R_q with small secrets and errors. We recall here a standard formulation adapted to our setting.

Definition 5.1 (Module-LWE distribution). Let $k, m \in \mathbb{N}$, and let χ_η be a distribution over R_q . An (n, k, m, η) -Module-LWE sample is a pair $(\mathbf{A}, \mathbf{b}) \in R_q^{m \times k} \times R_q^m$ where

$$\mathbf{A} \leftarrow R_q^{m \times k} \text{ is uniform, } \mathbf{s} \leftarrow \chi_\eta^{R_q^k}, \quad \mathbf{e} \leftarrow \chi_\eta^{R_q^m},$$

and

$$\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \in R_q^m.$$

We denote by $\mathcal{D}_{n,k,m,\eta}^{\text{MLWE}}$ the joint distribution of $(\mathbf{A}, \mathbf{b})$.

Definition 5.2 (Module-LWE problem). Adopt the notations in Definition 5.1. The decisional (n, k, m, η) -Module-LWE decision problem is: given a sample $(\mathbf{A}, \mathbf{b}) \in R_q^{m \times k} \times R_q^m$, distinguish with non-negligible advantage between:

- $(\mathbf{A}, \mathbf{b}) \leftarrow \mathcal{D}_{n,k,m,\eta}^{\text{MLWE}}$, and
- $(\mathbf{A}, \mathbf{b})$ chosen uniformly at random from $R_q^{m \times k} \times R_q^m$.

The corresponding search problem is to recover (some) secret $\mathbf{s}$ given $(\mathbf{A}, \mathbf{b})$ sampled from $\mathcal{D}_{n,k,m,\eta}^{\text{MLWE}}$. For a probabilistic polynomial-time adversary $\mathcal{B}$ against the decisional Module-LWE problem, define the advantage as:

$$\text{Adv}_{n,k,m,\eta}^{\text{MLWE}}(\mathcal{B}) := \left| \Pr[\mathcal{B}(\mathbf{A}, \mathbf{b}) = 1 : (\mathbf{A}, \mathbf{b}) \leftarrow \mathcal{D}_{n,k,m,\eta}^{\text{MLWE}}] - \Pr[\mathcal{B}(\mathbf{A}, \mathbf{b}) = 1 : (\mathbf{A}, \mathbf{b}) \leftarrow \mathcal{U}_{m,k}] \right|,$$

¹In the current reference implementation these roles are instantiated with the ICCS primitives (pseudoXOF, pseudoHash). Explicit domain-separation encoding across matrix generation, secret and error sampling, seed expansion, and key derivation would be fixed together with the standardized symmetric primitives that replace the placeholders in a subsequent round, as announced by the call [25].

where $\mathcal{U}_{m,k}$ denotes the uniform distribution over $R_q^{m \times k} \times R_q^m$.

Assumption 5.1 (Module-LWE hardness). For the parameter tuples (n, k, m, η) used in DKEM, no efficient classical or quantum algorithm can solve the underlying decisional (n, k, m, η) -Module-LWE problem with non-negligible advantage. Concretely, $\text{Adv}_{n,k,m,\eta}^{\text{MLWE}}(\mathcal{B})$ is negligible in the security parameter for any probabilistic polynomial-time adversary $\mathcal{B}$.

In particular, for m and k as in the parameter sets of Section 2.5, the problem resists all known lattice-reduction-based attacks up to the claimed concrete security levels. In the concrete instantiation of DKEM, we typically consider square matrices with $m = k$, i.e., $\mathbf{A} \in R_q^{k \times k}$, as in ML-KEM. However, the definitions above allow more general choices of (m, k) if desired.

5.1.3 Pseudorandomness of the DKE Shared Secret

This subsection proves the shared-secret pseudorandomness property of DKE. For an honestly generated initiator public key and an honest responder transcript, the responder's shared secret is computationally indistinguishable from a uniform string.

Definition 5.3. Let Π be a parameter set. The experiment $\text{Exp}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{A})$ samples a bit $b \leftarrow \{0, 1\}$, independent uniform coins $\text{coins}_A \leftarrow \{0, 1\}^n$, $\text{coins}_B \leftarrow \{0, 1\}^{2n}$, and an independent uniform string $U_n \leftarrow \{0, 1\}^n$. It computes

$$(pk, sk) \leftarrow \text{DKE.Initiate}(\text{coins}_A), \quad (c_1, ss) \leftarrow \text{DKE.Response}(pk, \text{coins}_B).$$

It then sets $K_b = ss$ if $b = 0$, and $K_b = U_n$ if $b = 1$, gives (pk, c_1, K_b) to $\mathcal{A}$, and runs $\mathcal{A}$ to output a bit b' . The experiment outputs 1 if and only if $b' = b$. The shared-secret pseudorandomness advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{A}) = \left| \Pr \left[\text{Exp}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{A}) = 1 \right] - \frac{1}{2} \right|,$$

where the probability is taken over the random coins of the experiment and of $\mathcal{A}$.

Two auxiliary facts are used in the proof. The first formalizes that deterministic public post-processing cannot increase computational distinguishing advantage. The second records the uniformity property delivered by unbiased reconciliation.

Lemma 5.1 (Deterministic public post-processing). Let X_0 and X_1 be random variables over a finite set $\mathcal{X}$, and let $f : \mathcal{X} \rightarrow \mathcal{Y}$ be a deterministic polynomial-time computable function. For every probabilistic polynomial-time distinguisher $\mathcal{D}$ distinguishing $f(X_0)$ from $f(X_1)$, there exists a probabilistic polynomial-time distinguisher $\mathcal{D}'$ distinguishing X_0 from X_1 such that

$$|\Pr[\mathcal{D}(f(X_1)) = 1] - \Pr[\mathcal{D}(f(X_0)) = 1]| = |\Pr[\mathcal{D}'(X_1) = 1] - \Pr[\mathcal{D}'(X_0) = 1]|.$$

Consequently, deterministic public transformations do not increase computational distinguishing advantage.

Proof. Define $\mathcal{D}'(x) = \mathcal{D}(f(x))$. Since f is deterministic and polynomial-time computable, $\mathcal{D}'$ is a probabilistic polynomial-time distinguisher. For each $b \in \{0, 1\}$, the experiment in which $\mathcal{D}'$ receives X_b is identical to the experiment in which $\mathcal{D}$ receives $f(X_b)$. Hence

$$\Pr[\mathcal{D}'(X_b) = 1] = \Pr[\mathcal{D}(f(X_b)) = 1].$$

The equality follows immediately. Taking the supremum over all probabilistic polynomial-time distinguishers gives the final claim. $\square$

Lemma 5.2. Adopt the hypotheses in Lemma 2.1. Let $x \leftarrow R_q$ be uniform. Sample $\mathbf{w} \leftarrow \text{Sig}(x)$ according to the protocol, and let $ss := \text{Rec}(x, \mathbf{w}) \in \{0, 1\}^n$. Then, for every $\mathbf{w}$ in the support of Sig , the conditional distribution of ss given $\mathbf{w}$ is uniform over $\{0, 1\}^n$. Consequently, $(\mathbf{w}, ss)$ is distributed identically to $(\mathbf{w}, U_n)$, where U_n denotes an independent uniform n -bit string.

Proof. The reconciliation and signal procedures are applied coefficient-wise. By Lemma 2.1, for each coefficient the output bit of Rec is uniform conditioned on the corresponding signal value. Since the coefficients are processed independently, the entire bit string ss is uniform conditioned on the full signal vector $\mathbf{w}$. $\square$

Theorem 5.3 (Shared-secret pseudorandomness of DKE). Let Π be a parameter set for DKE and DKEM, and let $\mathcal{A}$ be a probabilistic polynomial-time adversary against Definition 5.3. Then there exist probabilistic polynomial-time adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\text{Adv}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{A}) \leq \text{Adv}_{n,k,k,\eta}^{\text{MLWE}}(\mathcal{B}_1) + \text{Adv}_{n,k,k+1,\eta}^{\text{MLWE}}(\mathcal{B}_2).$$

In particular, under Assumption 5.1 and the unbiasedness property of the reconciliation function, the DKE shared secret is pseudorandom given the public key and the response transcript.

Proof. We use a sequence of games. For each game G_i , let p_i denote the probability that the game outputs 1, namely the probability that $\mathcal{A}$'s output bit b' equals the challenge bit b .

By Lemma 5.1, it is sufficient to prove the bound for the underlying response transcript before public deterministic compression: a distinguisher for $(\text{Compress}(\mathbf{u}_B, d_B), \mathbf{w})$ immediately gives a distinguisher for $(\mathbf{u}_B, \mathbf{w})$ with the same advantage. Hence, in the games below we write $c_1 = (\mathbf{u}_B, \mathbf{w})$; the same bound applies to the actual compressed transcript. The public matrix is generated as $\mathbf{A} = \text{GenMatrix}(\rho)$ from the public seed ρ . In the random-oracle model, this is the public uniform matrix distribution used in the Module-LWE assumption.

Game G_0 . This is the real experiment of Definition 5.3, written for the underlying uncompressed response transcript. The experiment samples $b \leftarrow \{0, 1\}$, $U_n \leftarrow \{0, 1\}^n$, and ρ , sets

$\mathbf{A} = \text{GenMatrix}(\rho)$, samples $\mathbf{s}_A, \mathbf{e}_A \leftarrow \chi_{\eta}^{R_q^k}$, and sets $pk = (\rho, \mathbf{t}_A)$ with $\mathbf{t}_A = \mathbf{A}\mathbf{s}_A + \mathbf{e}_A$. The responder samples $\mathbf{s}_B, \mathbf{e}_B \leftarrow \chi_{\eta}^{R_q^k}$ and $e'_B \leftarrow \chi_{\eta}^{R_q}$, and computes

$$\mathbf{u}_B = \mathbf{A}^T \mathbf{s}_B + \mathbf{e}_B, \quad k_B = \langle \mathbf{t}_A, \mathbf{s}_B \rangle + e'_B.$$

It samples $\mathbf{w} \leftarrow \text{Sig}(2k_B)$, sets $ss = \text{Rec}(2k_B, \mathbf{w})$, and gives $\mathcal{A}$ the tuple (pk, c_1, K_b) , where $c_1 = (\mathbf{u}_B, \mathbf{w})$, $K_b = ss$ if $b = 0$, and $K_b = U_n$ if $b = 1$. The game outputs 1 if and only if $b' = b$. Thus p_0 is the winning probability in Definition 5.3 for the underlying uncompressed transcript.

Game G_1 . This game is identical to G_0 , except that $\mathbf{t}_A$ in the public key is replaced by an independent uniform element of R_q^k . The responder uses this public $\mathbf{t}_A$ when computing k_B .

A reduction $\mathcal{B}_1$ embeds its (n, k, k, η) -Module-LWE challenge as the public-key pair $(\mathbf{A}, \mathbf{t}_A)$, programming $\text{GenMatrix}(\rho) = \mathbf{A}$, and simulates the rest of the game honestly. If the challenge is an MLWE sample, the simulation is G_0 ; if the challenge is uniform, the simulation is G_1 . Therefore

$$|p_0 - p_1| \leq \text{Adv}_{n,k,k,\eta}^{\text{MLWE}}(\mathcal{B}_1).$$

Game G_2 . This game is identical to G_1 , except that $(\mathbf{u}_B, k_B)$ is replaced by an independent uniform element of $R_q^k \times R_q$. The game still samples $\mathbf{w} \leftarrow \text{Sig}(2k_B)$, and in the $b = 0$ branch still sets $ss = \text{Rec}(2k_B, \mathbf{w})$.

In G_1 , the pair $(\mathbf{A}, \mathbf{t}_A)$ is uniform and independent of $\mathbf{s}_B$. Hence the tuple

$$\left(\begin{bmatrix} \mathbf{A}^T \\ \mathbf{t}_A^T \end{bmatrix}, \begin{bmatrix} \mathbf{A}^T \mathbf{s}_B + \mathbf{e}_B \\ \mathbf{t}_A^T \mathbf{s}_B + e'_B \end{bmatrix} \right)$$

has the form of an $(n, k, k+1, \eta)$ -Module-LWE sample with secret $\mathbf{s}_B$. A reduction $\mathcal{B}_2$ parses its $(n, k, k+1, \eta)$ -Module-LWE challenge as this stacked matrix-output pair, programs $\text{GenMatrix}(\rho) = \mathbf{A}$, and simulates the remaining steps honestly. If the challenge is an MLWE sample, the simulation is G_1 ; if the challenge is uniform, the simulation is G_2 . Therefore

$$|p_1 - p_2| \leq \text{Adv}_{n,k,k+1,\eta}^{\text{MLWE}}(\mathcal{B}_2).$$

Game G_3 . This game is identical to G_2 , except that the key given to $\mathcal{A}$ is an independent uniform string $U'_n \leftarrow \{0, 1\}^n$ in both branches $b = 0$ and $b = 1$.

We claim that $p_2 = p_3$. In G_2 , the value k_B is uniform over R_q and independent of $\mathbf{u}_B$. Since q is odd, $2k_B$ is also uniform over R_q . By Lemma 5.2, $\text{Rec}(2k_B, \mathbf{w})$ is uniform even conditioned on $\mathbf{w} = \text{Sig}(2k_B)$. Therefore, conditioned on $c_1 = (\mathbf{u}_B, \mathbf{w})$, the key in the $b = 0$ branch has the same distribution as an independent uniform n -bit string; the $b = 1$ branch already uses an

independent uniform U_n . Hence the views in G_2 and G_3 are identical, so $p_2 = p_3$. In G_3 , the tuple given to $\mathcal{A}$ is independent of b , and therefore $p_3 = 1/2$.

By Definition 5.3, the advantage of $\mathcal{A}$ is $|p_0 - \frac{1}{2}|$. Using $p_3 = 1/2$, the triangle inequality, the two Module-LWE transitions, and $p_2 = p_3$, we obtain

$$\begin{aligned} \text{Adv}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{A}) &= |p_0 - p_3| \\ &\leq |p_0 - p_1| + |p_1 - p_2| + |p_2 - p_3| \\ &\leq \text{Adv}_{n,k,k,\eta}^{\text{MLWE}}(\mathcal{B}_1) + \text{Adv}_{n,k,k+1,\eta}^{\text{MLWE}}(\mathcal{B}_2). \end{aligned}$$

This proves the theorem. $\square$

5.1.4 IND-CCA security of DKEM

We now prove the IND-CCA security of DKEM from the shared-secret pseudorandomness of DKE. The proof uses the transform exactly as specified in Algorithms 14 and 16: accepted outputs are derived from the DKE shared secret, while rejected outputs are derived from the rejection secret and the ciphertext tag.

Definition 5.4 (IND-CCA advantage of DKEM). Let Π be a DKEM parameter set. The game $G_{\Pi}^{\text{ind-cca}}(\mathcal{A})$ proceeds as follows. The adversary has oracle access to the domain-separated random oracles used by DKEM, including the seed-expansion oracle and the key-derivation oracle domains.

1. The challenger computes $(ek, dk) \leftarrow \text{DKEM.KeyGen}()$ and gives ek to $\mathcal{A}$.
2. The adversary may adaptively query the decapsulation oracle $\text{DKEM.Decaps}(dk, \cdot)$ on ciphertexts of its choice.
3. The challenger computes $(ct^*, K_0) \leftarrow \text{DKEM.Encaps}(ek)$, samples $K_1 \leftarrow \{0, 1\}^n$, samples $b \leftarrow \{0, 1\}$, and gives (ct^*, K_b) to $\mathcal{A}$.
4. The adversary may continue to query the decapsulation oracle on any ciphertext $ct \neq ct^*$.
5. The adversary outputs a bit b' . The game outputs 1 if $b' = b$, and outputs 0 otherwise.

The IND-CCA advantage of $\mathcal{A}$ is defined as:

$$\text{Adv}_{\text{DKEM}, \Pi}^{\text{ind-cca}}(\mathcal{A}) = \left| \Pr \left[G_{\Pi}^{\text{ind-cca}}(\mathcal{A}) = 1 \right] - \frac{1}{2} \right|.$$

Notation for the CCA bound. For the proof, write:

$$H_{\text{acc}}(ss) := \text{KDF}(ss; n), \quad H_{\text{rej}}(rej, c_2) := \text{KDF}(rej \| c_2; n),$$

for the two domain-separated KDF random-oracle domains in Algorithms 14 and 16. Let q_X bound adversarial seed-expansion queries on relevant inputs $(coins, \rho)$, where ρ is the public seed in ek . Let q_H and q_D bound adversarial queries to H_{acc} and to the decapsulation oracle. Internal oracle evaluations by the challenger are not counted.

We use two per-query error bounds. The value δ_{Π} is the reconciliation-failure bound from Section 3.4: for a table-generated ciphertext, it bounds the probability that decapsulation derives a DKE secret different from the responder secret stored with that entry, including under adaptive table queries. The value γ_{Π} is the fresh-recomputation bound. Namely, for a non-table query $ct = (c_1, c_2)$, decapsulation computes $ss_A \leftarrow \text{DKE.DeriveSecret}(c_1, sk)$ and $coins \leftarrow c_2 \oplus ss_A$. If the seed-expansion input $(coins, \rho)$ has not previously been evaluated by the adversary or internally by the challenger, then the next value of that random oracle is fresh; γ_{Π} bounds the probability that this fresh recomputation $\text{DKEM.Internal}(ek, coins)$ outputs exactly ct .

Lemma 5.4. Under Assumption 5.1, the fresh-recomputation bound satisfies

$$\gamma_{\Pi} \leq 2^{-\Omega(n)} + \text{negl}(n) = \text{negl}(n).$$

Consequently, $q_D \gamma_{\Pi}$ is negligible for every polynomially bounded q_D .

Proof. Fix an efficient execution prefix and a decapsulation query satisfying the freshness premise in the definition of γ_{Π} . At the moment of recomputation, the seed-expansion input $(coins, \rho)$ has not previously been evaluated, except by the current decapsulation call. Thus the random-oracle value $R \leftarrow \{0, 1\}^{2n}$ used by $\text{DKEM.Internal}(ek, coins)$ is sampled freshly and independently of the adversary's view. The queried ciphertext $ct = (c_1, c_2)$ is fixed before R is sampled.

A fresh non-table query can be accepted only if the recomputed DKE response transcript, including deterministic compression, equals the fixed transcript c_1 . We bound this transcript-hit probability. Apply the two Module-LWE hybrids from the proof of Theorem 5.3 to the transcript-only equality test that outputs 1 exactly when the fresh response transcript equals the fixed c_1 . Since the execution prefix and c_1 are efficiently generated before R is sampled, any non-negligible excess hit probability over the final hybrid, averaged over prefixes, would give such a Module-LWE distinguisher. Under Assumption 5.1, the real fresh transcript distribution is therefore computationally indistinguishable from the hybrid in which the responder value $\mathbf{u}_B$ and the reconciliation input k_B are uniform and independent.

In that uniform hybrid, the compressed responder component $\text{Compress}(\mathbf{u}_B, d_B)$ has maximum point probability $2^{-\Omega(n)}$: coefficient-wise compression maps independent uniform elements of R_q^k to a fixed positive number of bits per coefficient. Adding the signal vector $\mathbf{w}$ can only decrease the probability of hitting a fixed full transcript. Hence any fixed c_1 is hit with probability at most $2^{-\Omega(n)}$ in the uniform hybrid, and with probability at most $2^{-\Omega(n)} + \text{negl}(n)$ in the real distribution.

Since fresh acceptance implies this transcript hit, the per-query fresh-recomputation acceptance probability is $\gamma_\Pi \leq 2^{-\Omega(n)} + \text{negl}(n)$. A union bound over polynomially many decapsulation queries preserves negligibility. $\square$

Theorem 5.5 (IND-CCA reduction of DKEM). Let Π be a DKEM parameter set, and let $\mathcal{A}$ be a probabilistic polynomial-time adversary against Definition 5.4. Then there exists a probabilistic polynomial-time adversary $\mathcal{B}$ against Definition 5.3 such that

$$\text{Adv}_{\text{DKEM}, \Pi}^{\text{ind-cca}}(\mathcal{A}) \leq 2\text{Adv}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{B}) + q_D(\delta_\Pi + \gamma_\Pi) + \frac{q_X + q_H + q_D}{2^n}.$$

Proof. We work in the bit-guessing experiment of Definition 5.4. The challenge is generated at the challenge point in that experiment; before that point there is no forbidden challenge ciphertext. For each game G_i , let p_i denote the probability that the game outputs 1; in games without aborts, this is the probability that $\mathcal{A}$'s output bit equals the challenge bit.

Game G_0 . This is the real IND-CCA game. The challenger runs $(ek, dk) \leftarrow \text{DKEM.KeyGen}()$, answers first-stage decapsulation queries using $\text{DKEM.Decaps}(dk, \cdot)$, generates $(ct^*, K_0) \leftarrow \text{DKEM.Encaps}(ek)$ at the challenge point, samples $K_1 \leftarrow \{0, 1\}^n$ and $b \leftarrow \{0, 1\}$, and then answers all second-stage decapsulation queries except ct^* using real decapsulation. Thus p_0 is the winning probability in Definition 5.4.

Game G_1 . This game replaces real decapsulation by a table-based simulation. The challenger maintains a table $\mathcal{T}$ containing the relevant seed-expansion random-oracle entries and, for each entry $(coins, \rho)$, also stores the resulting tuple $(ct, coins, ss_B, K)$, where ss_B is the intermediate DKE shared secret produced inside Algorithm 14. Whenever the adversary queries seed expansion on such an input, the challenger answers consistently, runs $\text{DKEM.Internal}(ek, coins)$ with that oracle value, and stores the tuple if it is not already present. At the challenge point, the challenger samples $coins^*$; if the entry for $(coins^*, \rho)$ already exists, it uses the stored tuple as $(ct^*, coins^*, ss^*, K_0)$, and otherwise it computes and stores a fresh challenge tuple.

On a permitted decapsulation query $ct = (c_1, c_2)$, the simulated oracle returns the stored key K if ct appears in $\mathcal{T}$; otherwise it returns $H_{\text{rej}}(rej, c_2)$. The rejection domain is evaluated consistently in all games and is independent of the challenge bit.

The simulation can differ from real decapsulation only in two cases. If ct is a table-generated ciphertext, real decapsulation returns the stored key unless the DKE derivation gives $ss_A \neq ss_B$, which occurs with probability at most δ_Π per query. If ct is not table-generated but the recovered seed is already present in the seed-expansion table, recomputation gives the table ciphertext for that seed, so real decapsulation rejects ct as the simulator does. The only remaining case is a non-table ciphertext whose recovered seed is fresh; real decapsulation accepts such

a query with probability at most γ_Π . Therefore,

$$|p_0 - p_1| \leq q_D(\delta_\Pi + \gamma_\Pi).$$

Bad event. For a target pair (Z, coins^*) , define $\text{Bad}(Z, \text{coins}^*)$ in the table-based simulation, after the challenge coins are fixed, as the event that at least one of the following holds during the whole execution:

1. the adversary queries seed expansion on (coins^*, ρ) ;
2. the adversary queries H_{acc} at Z ;
3. a permitted decapsulation query is answered from $\mathcal{T}$ using a stored intermediate DKE shared secret equal to Z .

Internal oracle evaluations by the challenger do not trigger Bad . The first item includes seed-expansion queries made before the challenge point. The third item covers any non-challenge accepted ciphertext that would expose the same accepted-key oracle input as the challenge.

When Z is uniform, the mask $c_2^* = \text{coins}^* \oplus Z$ hides coins^* , and $H_{\text{acc}}(Z)$ reveals no information about Z unless the adversary queries that input. By deferred sampling, the first two items occur with probabilities at most $q_X/2^n$ and $q_H/2^n$. Each permitted decapsulation query can expose at most one stored accepted-key input, which equals the independent uniform value Z with probability at most 2^{-n} ; hence the third item contributes at most $q_D/2^n$. Therefore

$$\Pr[\text{Bad}(Z, \text{coins}^*)] \leq \frac{q_X + q_H + q_D}{2^n}$$

in the uniform- Z experiment.

To transfer this bound to the real challenge secret, construct a DKE shared-secret distinguisher that receives (ek, c_1^*, Z) , samples coins^* , sets $ct^* = (c_1^*, \text{coins}^* \oplus Z)$, and simulates the table-based CCA game for $\mathcal{A}$. At the challenge point it scans the transcript so far and outputs 1 if any of the three bad conditions has already occurred. After the challenge point, it halts and outputs 1 as soon as any bad condition occurs. Thus the simulator never has to answer a seed-expansion query on (coins^*, ρ) or an accepted-key query at Z consistently with the embedded challenge. If no bad condition occurs, the simulation is consistent. In the real branch this is the real challenge-secret distribution of G_1 ; in the uniform branch it is the uniform- Z experiment above. Since Definition 5.3 is stated as a bit-guessing advantage, this distinguishing gap is bounded by twice the advantage of some probabilistic polynomial-time adversary $\mathcal{B}$. Therefore,

$$\Pr[\text{Bad}(ss^*, \text{coins}^*)] \leq 2\text{Adv}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{B}) + \frac{q_X + q_H + q_D}{2^n}.$$

Game G_2 . This game is identical to G_1 , except that it halts and outputs an independent uniform bit if $\text{Bad}(ss^*, \text{coins}^*)$ occurs. The preceding bound gives:

$$|p_1 - p_2| \leq 2\text{Adv}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{B}) + \frac{q_X + q_H + q_D}{2^n}.$$

Game G_3 . This game is identical to G_2 , except that in the branch $b = 0$ the challenge key $K_0 = H_{\text{acc}}(ss^*)$ is replaced by an independent uniform string. If $\text{Bad}(ss^*, \text{coins}^*)$ does not occur, then the adversary never queries H_{acc} at ss^* , never queries the challenge seed, and never receives a permitted decapsulation answer derived from ss^* . Thus the random-oracle value $H_{\text{acc}}(ss^*)$ is exposed only as the challenge key, and replacing it by an independent uniform string is distribution-preserving. On the bad event both games already output an independent uniform bit. Hence $p_2 = p_3$.

In G_3 , the challenge key is uniform in both branches $b = 0$ and $b = 1$, and all oracle and decapsulation answers are independent of b . Therefore $p_3 = 1/2$. The triangle inequality gives

$$\begin{aligned} \text{Adv}_{\text{DKEM}, \Pi}^{\text{ind-cca}}(\mathcal{A}) &= \left| p_0 - \frac{1}{2} \right| \\ &\leq |p_0 - p_1| + |p_1 - p_2| + |p_2 - p_3| \\ &\leq q_D(\delta_\Pi + \gamma_\Pi) + 2\text{Adv}_{\text{DKE}, \Pi}^{\text{ss-pr}}(\mathcal{B}) + \frac{q_X + q_H + q_D}{2^n}. \end{aligned}$$

This proves the theorem. □

Let Π be an admissible asymptotic instantiation of DKEM, and let all negligibility statements in this paragraph be with respect to the security parameter. Assume that, for this choice of parameters, the Module-LWE advantages appearing in Theorem 5.3 are negligible, the correctness-failure bound δ_Π is negligible, and the fresh-check bound γ_Π from Lemma 5.4 is negligible. Then Theorem 5.3 makes the DKE shared-secret pseudorandomness term in Theorem 5.5 negligible. Moreover, for any adversary making polynomially many queries, the term $q_D(\delta_\Pi + \gamma_\Pi)$ is negligible, and the term $(q_X + q_H + q_D)/2^n$ is negligible whenever 2^{-n} is negligible for the chosen parameter family. Therefore, under the above parameter conditions, DKEM is IND-CCA secure in the classical random-oracle model.

5.2 Analysis with respect to known attacks

5.2.1 Attacks against the underlying MLWE problem

The best known attacks against the underlying MLWE problem in DKEM do not significantly exploit the algebraic structure of module lattices. Several works have investigated whether the additional algebraic structure present in Ring-LWE and Module-LWE instances can be leveraged to improve attacks. More concretely, the works in [36, 37] propose quantum algorithms for recovering short generators of principal ideals in cyclotomic rings, which translates to efficient

algorithms for Ideal-SVP in certain settings. Similarly, [38] gave efficient quantum algorithms for the principal ideal problem in arbitrary degree number fields. However, subsequent analysis established that these results do not extend to the module lattice setting in any practically threatening way. In particular, as noted in [17], any attempt to use the reduction in [39] from MLWE to Ring-LWE to attack DKEM would produce an Ring-LWE instance with a substantially larger modulus and error, making the resulting problem harder to attack than the original MLWE instance. Most recently, theoretical work on module-BKZ has shown that exploiting module structure can yield subexponential speedups for certain cyclotomic fields [40, 41]; however, this speedup applies only to non-power-of-two cyclotomic fields, so the ring used in DKEM is explicitly outside the scope of this result. While all these algebraic techniques lead to a modest reduction in the estimated cost of certain attacks, the improvement is limited and does not currently lead to practical attacks against standard parameter sets [29]. We therefore analyze the hardness of the MLWE problem as an LWE problem.

Attacks against the LWE problem. Many algorithms exist for solving the LWE problem [42]. However, most combinatorial and algebraic approaches require significantly more samples than are available in the present scheme, which provides only $m = (k + 1)n$ LWE samples. In particular, BKW-type attacks typically need thousands of samples to succeed [43] and are therefore effectively ruled out in this limited-sample regime. Similarly, algebraic attacks such as the Arora–Ge linearization method and Gröbner basis techniques require substantially more samples than available here and are therefore infeasible [44]. Consequently, the relevant attacks are lattice-reduction–based methods using BKZ [30, 31], namely the primal and dual attacks (and later hybrid attacks), which form the primary basis for estimating the hardness of our LWE problem.

The BKZ algorithm works by iteratively calling an oracle that solves the SVP in a reduced sublattice of block size b , and uses the result to improve the basis of the parent lattice. The number of calls to the SVP oracle is known to remain polynomial [45]. As a first simple approach we introduce the core-SVP hardness methodology, which ignores this polynomial factor and considers only the cost of a single SVP oracle call in dimension b [26]. The results of this analysis can be seen in Table 6. While this approach provides useful intuition, it is too coarse to yield accurate security estimates in the classical framework [17]. More precise concrete estimates are therefore provided in Section 5.2.2.

Enumeration vs sieving. Two algorithmic approaches are used for the SVP oracle in BKZ: enumeration and lattice sieving. Enumeration algorithms systematically explore a bounded search tree of candidate vectors in the lattice basis in order to find a shortest vector. They require relatively little memory, but have running time that grows super-exponentially in the block size b . They are thus only useful for small block dimensions. Lattice sieving algorithms instead maintain large lists of lattice vectors and iteratively combine them to produce shorter

vectors. They have exponential complexity both in memory and time, and outperform enumeration around dimension 70 [46]. Modern sieving techniques based on locality-sensitive hashing and filtering achieve a heuristic classical time complexity of approximately $2^{0.292b+o(b)}$ [47]. This last value is the one we considered for the core-SVP hardness estimations. Quantum algorithms can further accelerate lattice sieving. Applying Grover-type search techniques reduces the complexity to approximately $2^{0.265b+o(b)}$ [48, 49], and more recent work using quantum hypercone locality-sensitive filtering improves this bound to roughly $2^{0.257b+o(b)}$ [50, 51]. For the core-SVP hardness estimations we adopted this latter value, corresponding to a stronger quantum attacker assumption and therefore a more conservative bound on our quantum security.

Primal and dual lattice attacks. Following the core-SVP methodology, the hardness of the LWE instance is estimated by determining the block size b for which BKZ reduction is expected to succeed. Two main lattice attacks are considered in this framework: the *primal attack* and the *dual attack*.

Primal attack. The primal attack embeds the LWE instance into a lattice in which the vector containing the secret and error terms appears as a uniquely short vector. Given an LWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e})$, one constructs the lattice $\Lambda = \{\mathbf{x} \in \mathbb{Z}^{m+kn+1} : (\mathbf{A} \mid \mathbf{I}_m \mid -\mathbf{b})\mathbf{x} \equiv \mathbf{0} \pmod{q}\}$ which has dimension $d = m + kn + 1$ and volume q^m . This lattice contains the vector $\mathbf{v} = (\mathbf{s}, \mathbf{e}, 1)$ whose norm is approximately $\|\mathbf{v}\| \approx \sigma\sqrt{kn+m}$, where σ denotes the standard deviation of the secret and error coefficients. The number of used samples m is numerically optimized among the choices from 0 to $(k+1)n$, together with the BKZ block size. The success of BKZ is modeled using the geometric series assumption (GSA), producing a basis whose Gram–Schmidt norms follow approximately $\|\mathbf{b}_i^*\| = \delta^{d-2i-1} \cdot \text{Vol}(\Lambda)^{1/d}$, where $\delta = \left((\pi b)^{1/b} \cdot b/2\pi e\right)^{1/2(b-1)}$. The short vector $\mathbf{v}$ is expected to be detected by BKZ once its projection onto the subspace spanned by the last b Gram–Schmidt vectors becomes shorter than $\|\mathbf{b}_{d-b}^*\|$, that is, the attack is successful if and only if $\sigma\sqrt{b} \leq \delta^{2b-d-1} \cdot q^{m/d}$ [49, 26, 17]. This corresponds to the unique SVP (uSVP) formulation. Nevertheless, in addition to that, the primal attack can also be viewed as a bounded distance decoding (BDD) problem. After lattice reduction, the target vector lies close enough to the lattice and can be recovered using enumeration if it is within the decoding radius. In practice, both attack variants are considered, and the minimum resulting cost is used as the primal attack cost estimate, which for our parameters is generally the BDD.

Dual attack. The dual attack aims to distinguish LWE samples from uniform by exploiting a statistical bias, so its goal is to find a short vector $\mathbf{w} = (\mathbf{x}, \mathbf{y})$ of length ℓ in the dual lattice $\Lambda^* = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^m \times \mathbb{Z}^{kn} : \mathbf{A}^T \mathbf{x} \equiv \mathbf{y} \pmod{q}\}$, which has dimension $d = m + kn$ and volume q^{kn} . In this case, $z = \mathbf{x}^T \mathbf{b} = \mathbf{y}^T \mathbf{s} + \mathbf{x}^T \mathbf{e} \pmod{q}$ follows a Gaussian distribution of standard

deviation $\ell\sigma$ if $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ is an LWE sample and is uniform modulo q otherwise. The maximal statistical distance between these two distributions is bounded by $\varepsilon = 4\exp(-2\pi^2\tau^2)$, where $\tau = \ell\sigma/q$, so a vector of length ℓ yields a distinguishing advantage ε against decision-LWE. Under GSA assumption $\ell \approx \|\mathbf{b}_0\| = \delta^{d-1}q^{kn/d}$, for a vector produced by BKZ. Therefore, obtaining an ε -distinguisher requires running BKZ with block size b such that $-2\pi^2\tau^2 \geq \ln(\varepsilon/4)$. To obtain a constant success probability, the advantage must be amplified by repeating the procedure approximately $1/\varepsilon^2$ times. Since sieving algorithms are expected to output about $2^{0.2075b}$ sufficiently short vectors, the total repetition cost is estimated as $R = \max\left(1, \frac{1}{2^{0.2075b\varepsilon^2}}\right)$. This estimate assumes conservatively for the defender that all vectors returned by the sieving algorithm are as short as the shortest one [26, 17].

Hybrid attacks. Both the primal and the dual attacks have hybrid variants, which combine lattice reduction techniques with an exhaustive search over a subset of the secret. In a hybrid attack, the secret vector is decomposed into two parts, one of which is guessed, while the remaining part is recovered via a reduced lattice problem of lower dimension. This trade-off allows the attacker to balance the combinatorial cost of guessing against the cost of lattice reduction. For the primal setting, hybrid variants do not typically yield a significant improvement over standard primal attacks, since the structure of the corresponding uSVP instance does not benefit as strongly from partial secret guessing. In contrast, hybrid dual attacks can provide a noticeable improvement over pure dual attacks when the secret is small, as is our case. The best known attacks against such schemes are therefore hybrid dual attacks, which optimize over the number of guessed secret components and then apply a dual lattice reduction to the resulting reduced instance [52, 53, 54].

5.2.2 Refined gate-count estimate

The core-SVP hardness methodology is too coarse to produce accurate concrete security estimates in the classical setting. To obtain refined estimates, we rely on the Lattice Estimator of Albrecht et al. [55], which is the most widely accepted tool in the cryptographic community for this purpose.

The estimator models lattice reduction using progressive BKZ, in which the BKZ block size increases gradually throughout the reduction process rather than being fixed from the start. This is generally considered more realistic than fixed-block BKZ analyses. We configure the estimator to use the Gama–Nguyen basis shape simulator (GN11) [56] in place of GSA to model the Gram–Schmidt profile of the reduced basis. Unlike GSA, GN11 is calibrated from experimental BKZ data and captures tail effects in the Gram–Schmidt norms that GSA systematically misses. This provides more accurate predictions of BKZ behavior at the large block sizes relevant to cryptographic security estimates.

For the sieving cost model, we use the MATZOV reduction cost model [57] as implemented

Parameter set	Attack	Lattice dimension d	Mode	BKZ blocksize β	$\log_2(\text{gates})$
DKEM-128	Primal	1014	Classical	414	120.9
			Quantum	414	106.4
	Dual	1024	Classical	424	123.8
			Quantum	424	109.0
	Dual hybrid	1024	Classical	395	115.3
			Quantum	401	103.1
DKEM-256	Primal	1930	Classical	890	259.9
			Quantum	890	228.7
	Dual	1942	Classical	904	264.0
			Quantum	904	232.3
	Dual hybrid	1942	Classical	826	241.2
			Quantum	838	215.4
DKEM-512	Primal	3686	Classical	1881	549.3
			Quantum	1881	483.4
	Dual	3834	Classical	1890	551.9
			Quantum	1890	485.7
	Dual hybrid	3784	Classical	1732	505.7
			Quantum	1753	450.5

Table 6: Core-SVP hardness of the schemes (using ADPS16 [42] and ChaLoy21 [51] models for classical and quantum estimates respectively in the Lattice Estimator [55, 24]).

in the estimator. The estimator automatically accounts for all BKZ reduction costs, progressive overheads, and dimension reductions (dimensions-for-free [58]). No additional multiplicative constants or overhead factors are applied manually, unlike earlier analyses such as in [17].

In the original Kyber round-3 security analysis [17], the primal attack was identified as the dominant attack, with the dual attack providing a secondary reference point at comparable or slightly higher cost. Since then, the landscape of dual attacks has evolved significantly. Guo and Johansson [59] and, independently, the MATZOV team [57] proposed substantial improvements to the dual attack by incorporating a Fast Fourier Transform (FFT) step into the dual-sieve framework, claiming reductions of 5–15 bits in the concrete security of Kyber parameter sets, with potentially similar impact on comparable parameter sets. These results attracted significant attention as they appeared to threaten the security margins of several NIST candidates. However, Ducas and Pulles [60] showed that a key *independence heuristic* underlying these improved dual attacks leads to incorrect predictions in parameter regimes directly relevant to cryptographic applications. Their analysis suggests that the primal attack likely regains the status of the most efficient classical attack under a corrected model. Carrier et al. subsequently proposed a rigorous variant of the MATZOV dual attack in [61], replacing modulus switching with an efficient decoding algorithm based on polar codes over $\mathbb{Z}_q$, and confirmed the claimed improvements under this corrected framework, partially restoring the MATZOV security reductions.

Given this landscape, we adopt the MATZOV reduction cost model [57] as a conservative lower bound on the security of DKEM. We emphasize that the independence heuristic underlying the MATZOV dual attack has been called into question in [60], but the corrected analysis in [61] yields results of comparable magnitude under a rigorous foundation. The primal attack costs are not affected by this controversy at all.

Parameter Set	Attack	Lattice dimension d	BKZ blocksize β	$\log_2(\text{gates})$
DKEM-128	Primal	1009	396	142.1
	Dual	1024	424	149.9
	Dual hybrid	1024	387	139.7
DKEM-256	Primal	1914	870	275.4
	Dual	1954	918	288.5
	Dual hybrid	1954	823	262.3
DKEM-512	Primal	3674	1855	552.9
	Dual	3866	1930	574.0
	Dual hybrid	3836	1736	519.9

Table 7: Refined classical security estimates using the MATZOV reduction cost model [57] and GN11 basis shape simulator [56], as implemented in the Lattice Estimator [55, 24].

The resulting estimates are summarized in Table 7. They represent a lower bound on the true classical security level: the true cost to an adversary is at least as large as reported, and likely larger under more optimistic models for the defender. As shown, the most cost-efficient attack across all parameter sets is the dual-hybrid attack. In this case, the proposed parameter sets achieve classical security levels of 139.7, 262.3, and 519.9 bits for DKEM-128, DKEM-256, and DKEM-512 respectively. These margins comfortably exceed the target security levels of 128, 256, and 512 bits [25]. Under less aggressive attack models the true security margins are expected to be higher.

Regarding lattice reduction costs, the most demanding case (DKEM-128) requires BKZ with block size $\beta = 387$. For comparison, the current record for solving SVP instances as of January 2026 is dimension 210 [62], which remains significantly below the block sizes required to attack any DKEM parameter set.

All reported costs are heuristic and rely on assumptions about lattice reduction behavior, SVP oracle complexity, and hybrid attack efficiency. The resulting estimates differ from earlier analyses such as [17], which did not account for hybrid attacks, relied on less refined cost models, and did not incorporate the developments in dual attack methodology described above.

5.2.3 About quantum security

Estimating the concrete security of lattice-based constructions against quantum adversaries remains an open problem, and no community consensus currently exists on a definitive refined cost model analogous to those available for classical attacks. We therefore adopt a core-SVP-based approach, following standard practice in analyses of schemes such as ML-KEM [17],

modeling the cost of the quantum SVP oracle as 2^{cb} for block size b . A natural choice is $c = 0.265$, corresponding to the cost of Grover-accelerated lattice sieving [48, 49]. In this work, however, we adopt the slightly tighter value $c = 0.257$ from Chailloux and Loyer [51], which uses quantum random walks instead of plain Grover search and represents a more conservative (attacker-favorable) security assumption. In this setting, the quantum security bits are 103.1, 215.4 and 450.5 for DKEM-128, DKEM-256 and DKEM-512 respectively (see Table 6). These values significantly exceed the target levels of 80, 128, and 256 quantum security bits [25].

5.2.4 Attacks against symmetric primitives

The symmetric primitives currently used in the reference implementation of DKEM are SM3-based constructions provided by the NGCC auxiliary function library, specifically `sm3hash`, `pseudoXOF`, and `pseudohash`.

The NGCC documentation explicitly states that `pseudoXOF` and `pseudohash` are provided solely for correctness verification in the current evaluation round and do not guarantee security, and that they will be replaced by standardized primitives in subsequent rounds [25]. The security analysis of DKEM with respect to its symmetric components is therefore conditional: we assume that the final ICCS-standardized replacements will satisfy the properties required by the security proof, namely pseudorandomness for the XOF and PRF components, and collision resistance for the hash components.

Regarding SM3 itself, no attack on the full construction is currently known. Known cryptanalytic results are limited to step-reduced variants and do not threaten the full 64-round function [63, 64]. The security proofs model the hash and XOF functions as random oracles. Should the ICCS replacement primitives be based on a different underlying function, this analysis will need to be updated accordingly.

5.2.5 Attacks exploiting reconciliation failures

In Theorem 5.5 we see that reconciliation failure probability δ plays a role in the attacker’s advantage. A first naive approach for an attacker would be to try random ciphertexts. This has a success probability of $q_d \delta$, where q_d is the number of decapsulation queries. With up to 2^{80} ciphertext queries (NGCC adversary model [25]), the probability of triggering at least one failure is at most $2^{-52.7}$, $2^{-101.2}$ and $2^{-87.0}$ for DKEM-128, DKEM-256 and DKEM-512 respectively, which is negligible.

A more sophisticated adversary wishing to exploit failures proceeds as follows: rather than submitting random ciphertexts, the adversary searches offline for values of *coins* (Line 1 of Alg. 15) such that the derived randomness r produces response-side noise vectors with unusually large norm. In fact, larger response-side noise increases the magnitude of the inner products involved in reconciliation, raising the chance that the reconciliation threshold is exceeded and

a failure occurs. This strategy is known as *failure boosting* [65] and is especially useful when the adversary is constrained to the number of decapsulation queries (e.g. 2^{80} in the NGCC model [25]), or has access to a quantum computer (since Grover’s algorithm can speed up this offline search quadratically). However, this search faces two compounding obstacles. First, since XOF behaves as a random oracle, there is no shortcut: finding coins that produce large-norm noise requires an exhaustive search, and high-dimensional noise vectors are extremely unlikely to deviate significantly from their expected size, so the search remains exponentially expensive even with Grover’s quadratic speedup. Second, even if the adversary finds larger noise, the actual failure also depends on the initiator’s secret key noise, which is unknown, so the improvement in failure probability per query is only modest. The gain from finding large noise is therefore far outweighed by the cost of finding it, for all our parameter sets [65, 66, 17].

From one to multiple failures. As explained in [17], an attacker might need to trigger more than one failure to mount a key recovery attack. The security of the scheme, however, must not depend on the difficulty of triggering many failures, since the work in [67] showed that once an adversary finds a single failing ciphertext, finding further ones becomes dramatically cheaper, because the first failure reveals information about the direction of the secret key noise, which can be exploited to craft ciphertexts that are much more likely to fail. The implication is that the security thus relies on whether the adversary can trigger even one failure. Our parameter sets are chosen precisely so that this is computationally infeasible.

Multitarget attacks. An adversary targeting many users simultaneously might precompute some failure-inducing coins and attempt to reuse them across many victims [68]. Nevertheless, in line 2 of Alg. 14 these kind of attacks are prevented, since ρ (the last n bits of the encapsulation key) is hashed into the randomness. A different user’s key embeds a different ρ , so the same *coins* value produces a completely independent r . This is the direct analogue of ML-KEM’s protection via hashing the public key into the encapsulation randomness [17, 8].

5.3 Other security properties

The NGCC submission requirements encourage analysis of additional security properties beyond IND-CCA security. We address here the most relevant properties of DKEM: side-channel resistance, forward secrecy, security under (temporary) key reuse, multi-key/multi-target security, and robustness against reconciliation failure.

Side-channel resistance. DKEM is designed for constant-time implementation on all target platforms. The performance-critical, secret-dependent operations (NTT-based polynomial arithmetic, centered-binomial sampling, modular reduction, compression and serialization) are inherited directly from the constant-time, public-domain ML-KEM code bases (PQClean [12],

mlkem-native [14]) and from the constant-time Plantard arithmetic of [15], none of which use secret-dependent branches, memory indices, or variable-time divisions. The components specific to DKEM are implemented with the same discipline in all our implementations. The two-layer structure delegates all long-term-secret-dependent computation to the DeriveSecret step, which simplifies side-channel review and localizes the future addition of masking.

Forward secrecy. As a key-encapsulation mechanism, DKEM provides forward secrecy when deployed with ephemeral key pairs: a fresh (ek, dk) generated for a single key-establishment transaction and destroyed afterwards ensures that later compromise of long-term material cannot expose past session keys. DKEM is particularly well suited to this mode. Key generation costs no more than the corresponding ML-KEM operation, and the CPA core DKE is itself a complete two-pass ephemeral key exchange. As shown in [10], CPA-secure KEMs already suffice for ephemeral key establishment in protocols such as TLS 1.3; DKEM therefore supports both ephemeral deployment, with the lightweight DKE core providing forward secrecy by itself, and static-key deployments, where the IND-CCA security of DKEM (Section 5.1) permits safe key reuse.

Security under (temporary) key reuse. The IND-CCA security established in Section 5.1 is precisely the property required for safe key reuse: an adversary with adaptive access to a decapsulation oracle under a fixed long-term key gains no advantage, so a DKEM key may be reused without loss of security. The underlying CPA core DKE, like all reconciliation- and LPR-type CPA schemes, must by contrast be restricted to ephemeral use: reuse of a static DKE key exposes it to key-mismatch attacks [34, 33, 32], in which the publicly transmitted signal acts as a reconciliation oracle. DKEM is the recommended choice whenever keys may be reused.

Multi-key and multi-target security. DKEM incorporates an explicit multi-target countermeasure: the public seed ρ (the last n bits of the encapsulation key) is folded into the encapsulation randomness (Line 2 of Algorithm 14), so that the same *coins* value expands to independent randomness for distinct users. This prevents any offline work across users and thereby neutralizes certain kinds of multi-target attacks, including multi-target failure boosting. It is the direct analogue of ML-KEM’s hashing of the public key into the encapsulation randomness [17, 8], achieved here at lower cost since only ρ , rather than a full public-key digest, is bound (see Section 5.2.5). At the assumption level, the underlying Module-LWE problem retains its hardness in the multi-user setting up to the standard hybrid loss linear in the number of users, so no structural multi-key weakness arises.

Robustness against reconciliation failure and reaction attacks. The reconciliation-failure probability is cryptographically negligible for every parameter set ($\leq 2^{-132.7}$). Together with

implicit rejection, this leaves the adversary no failure or validity signal to exploit. As analyzed in Section 5.2.5, failure boosting and single-failure key-recovery strategies are therefore infeasible within the NGCC query model.

Symmetric-layer side channels. Because the symmetric layer is built entirely on SM3, DKEM can draw directly on the hardware and constant-time SM3 implementations available in the commercial cryptography ecosystem, reducing the side-channel exposure of hashing, key derivation, and pseudorandom expansion on deployed platforms.

6 Feature Statements

This section provides explicit feature statements for DKEM against each of the criteria defined by the NGCC [25]: innovativeness, simplicity, flexibility, compatibility, extensibility, and performance advantages on variant platforms. Each feature is argued from specific verifiable design decisions documented in the preceding sections.

Innovativeness The central innovation of DKEM is its construction lineage. Every MLWE KEM in current international standards builds a shared secret using the LPR framework: the encapsulator embeds a random message in a noisy ciphertext, the decapsulator recovers the message via approximate decryption, and the session key is derived. DKEM does not encrypt a message: both parties instead compute approximations to the same lattice product from their respective MLWE samples and use a signal function to extract one bit of shared secret per coefficient, adapting the approach in [5, 6] to MLWE.

The second major innovation is the tag based Fujisaki-Okamoto transform. As no message is encrypted, DKEM’s core has no plaintext to recover, so the standard FO template does not apply [9]. In order to achieve CCA security, the tag-based FO transform addresses this by treating the encapsulation randomness as the transported artifact, masking it with the CPA shared secret in a tag appended to the ciphertext. The decapsulator recovers the randomness from the tag and re-runs encapsulation to verify ciphertext integrity.

Moreover, the reconciliation mechanism of [6] is adapted to the module setting and tuned to the ML-KEM ring. Transmitting ℓ bits per coefficient over 2^ℓ signal subintervals to trade a small amount of bandwidth for a markedly lower failure probability is the general *key-consensus* approach studied in [18, 19] (and, for $\ell = 1$, the reconciliation of [5, 6, 26]). DKEM’s contribution here is the instantiation of this mechanism on the standardized ML-KEM parameters, together with a self-contained proof (Lemma 2.1) that the extracted bit is perfectly biased given the signal under our randomized-doubling step.

Simplicity. DKEM uses a single noise parameter η across all secret and error vectors at every security level, in contrast to ML-KEM’s two-parameter design. This unifies the sampling into a

single CBD_η instantiation, reduces code footprint and hardware area, and eliminates a class of implementation errors arising from applying the wrong parameter in the wrong context.

Flexibility. DKEM provides three parameter sets covering classical security strengths of at least 128, 256, and 512 bits with corresponding quantum-resistant security of at least 80, 128, and 256 bits, spanning the full range mandated by the Submission Requirements [25]. All three share the same construction, only differing in the choice of the parameters.

As requested by NGCC, *algorithms can be implemented securely and efficiently on a wide variety of platforms, supporting various tradeoffs between efficiency and cost.* Concretely, in contrast to more costly Gaussian sampling, DKEM samples secret and noise polynomials using CBD, which can be implemented securely and efficiently across various platforms [26]. Besides, as demonstrated by existing ML-KEM/ML-DSA optimizations, the underlying polynomial arithmetic in DKEM can likewise be efficiently adapted to a broad range of platforms, from high-performance platforms [69, 70, 71] to resource-constrained devices [15, 72, 73]. Furthermore, DKEM can support various time-memory trade-offs on highly constrained IoT devices [74, 75, 76], providing substantial flexibility for secure and efficient deployment across diverse platforms. All three security levels are supported from a shared codebase with compile-time selection, minimizing deployment and maintenance burden.

Compatibility. The symmetric layer of DKEM (hash function, XOF, KDF, and rejection hash) is based entirely on SM3, making DKEM natively compatible with the Chinese SM cryptographic suite at the specification level.

Furthermore, being based on the MLWE problem, DKEM shares the same mathematical foundation as the dominant PQC standardized schemes. Moreover, for the 128 and 256-bit parameter sets, DKEM uses the same ring, modulus and rank as ML-KEM-512 and ML-KEM-1024, inheriting also the NTT structure. All this allows existing ML-KEM implementations to contribute, and enables existing research and cryptanalysis exposure to apply to this scheme.

Extensibility. DKE, the CPA core of DKEM, is a complete two-party key exchange protocol, where both parties derive the same n -bit shared secret. Thus, inside this same specification there is a whole complete CPA-secure key exchange protocol that could be used for any case where CCA security is not a requirement.

Moreover, the structure of DKE extends naturally to authenticated key exchanges. Two of these possibilities are submitted as separate submissions in this NGCC call, namely **ADKEX** and **DKEX**.

On the other hand, a structureless-LWE variant can also be obtained by replacing the module polynomial arithmetic with plain-LWE matrix operations while retaining the signal function, mod-2 extractor, and tag-based FO transform unchanged, providing an assumption-diversity extension consistent with the NGCC emphasis on hard-problem diversity.

Performance Advantages on Variant Platforms. The reference implementation executes the complete DKEM scheme using portable C, while the performance optimization version uses an x86-64 AVX2 backend for the performance-critical polynomial-arithmetic operations, including modular arithmetic, NTT and inverse NTT, base multiplication, and matrix–vector accumulation. The measured x86 results show mean execution-time reductions ranging from approximately 15.3 percent to 52.5 percent across KeyGen, Encaps, and Decaps for the three parameter sets, without changing the public-key, secret-key, ciphertext, or shared-secret sizes. For resource-constrained platforms, DKEM also includes a Cortex-M4 implementation using optimized Plantard arithmetic and an assembly-optimized SM3 backend. The additional QEMU evidence covers all three parameter sets and reports operation-level cycle counts for KeyGen, Encaps, and Decaps. These implementations provide evidence of instruction-set acceleration on x86-64 and functional execution on a Cortex-M4 target.

Furthermore, to facilitate a fair comparison with the NIST PQC standard ML-KEM, when replacing the existing SM3-based pseudorandom number generator with SHA3, consistent with ML-KEM, the proposed performance-optimized implementations of DKEM-128 and DKEM-256 slightly outperform ML-KEM-512 and ML-KEM-1024 on both x86 and Cortex-M4, demonstrating performance advantages on various platforms.

References

- [1] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
- [2] Oded Regev. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing (STOC)*, pages 84–93. ACM, 2005.
- [3] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On Ideal Lattices and Learning with Errors over Rings. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010.
- [4] Adeline Langlois and Damien Stehlé. Worst-Case to Average-Case Reductions for Module Lattices. *Designs, Codes and Cryptography*, 75:565–599, 2015.
- [5] Jintai Ding, Xiang Xie, and Xiaodong Lin. A Simple Provably Secure Key Exchange Scheme Based on the Learning with Errors Problem. Cryptology ePrint Archive, Paper 2012/688, 2012.
- [6] Jintai Ding, Tsuyoshi Takagi, Xinwei Gao, and Yuntao Wang. Ding Key Exchange. NIST’s call for Post-quantum Cryptography Standardization, 2017.

- [7] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS – Kyber: a CCA-secure module-lattice-based KEM. Cryptology ePrint Archive, Paper 2017/634, 2017.
- [8] National Institute of Standards and Technology. FIPS 203: ML-KEM: Module Lattice-Based Key-Encapsulation Mechanism. <https://doi.org/10.6028/NIST.FIPS.203>, 2024. U.S. Department of Commerce, August 2024.
- [9] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A Modular Analysis of the Fujisaki-Okamoto Transformation. Cryptology ePrint Archive, Paper 2017/604, 2017.
- [10] Biming Zhou, Haodong Jiang, and Yunlei Zhao. CPA-secure KEMs are also sufficient for Post-Quantum TLS 1.3. Cryptology ePrint Archive, Paper 2024/1360, 2024.
- [11] Peter Schwabe, Douglas Stebila, and Thom Wiggers. More efficient post-quantum KEMTLS with pre-distributed public keys. Cryptology ePrint Archive, Paper 2021/779, 2021.
- [12] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving Software Quality in Cryptography Standardization Projects. In *IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6-10, 2022*, pages 19–30, Los Alamitos, CA, USA, 2022. IEEE Computer Society.
- [13] Open Quantum Safe project. liboqs: C library for prototyping and experimenting with quantum-resistant cryptography. <https://github.com/open-quantum-safe/liboqs>, 2026.
- [14] Hanno Becker, Matthias J. Kannwischer, and The mlkem-native project authors. mlkem-native: A secure, fast, and portable C90 implementation of ML-KEM / FIPS 203. <https://github.com/pq-code-package/mlkem-native>, 2026. Version v1.1.0. Accessed: 2026-06-23.
- [15] Junhao Huang, Jipeng Zhang, Haosong Zhao, Zhe Liu, Ray C. C. Cheung, Çetin Kaya Koç, and Donglong Chen. Improved Plantard arithmetic for lattice-based cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(4):614–636, 2022.
- [16] Vadim Lyubashevsky and Gregor Seiler. NTTRU: Truly Fast NTRU Using NTT. Cryptology ePrint Archive, Paper 2019/040, 2019.
- [17] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation (Version

- 3.02). Submission to the NIST Post-Quantum Cryptography Standardization Project, Round 3, 2021. Updated August 4, 2021.
- [18] Zhengzhong Jin and Yunlei Zhao. Optimal Key Consensus in Presence of Noise. Cryptology ePrint Archive, Paper 2017/1058, 2017.
 - [19] Yunlei Zhao, Zhengzhong Jin, Boru Gong, and Guanye Sui. KCL (pka OKCN/AKCN/CNKE): Key Consensus from Lattice. Submission to NIST PQC Standardization Project, Round 1.
 - [20] James W. Cooley and John W. Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 19(90):297–301, 1965.
 - [21] W. Morven Gentleman and Gordon Sande. Fast Fourier Transforms: For Fun and Profit. In *Proceedings of the November 7–10, 1966, Fall Joint Computer Conference (AFIPS '66)*, pages 563–578, 1966.
 - [22] Patrick Longa and Michael Naehrig. Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography. In *Cryptology and Network Security (CANS 2016)*, volume 10052 of *Lecture Notes in Computer Science*, pages 124–139. Springer, 2016.
 - [23] Gregor Seiler. Faster AVX2 optimized NTT multiplication for Ring-LWE lattice cryptography. Cryptology ePrint Archive, Paper 2018/039, 2018.
 - [24] *Basque Centre for Applied Mathematics*. DKE Scripts: Failure-Probability, Communication-Cost and Concrete-Security Analysis Tools for DKEM. GitHub repository, 2026. Accessed 2026-06-24.
 - [25] Institute of Commercial Cryptography Standards. Announcement on Launching the Next-generation Commercial Cryptographic Algorithms Program (NGCC), October 2025.
 - [26] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum Key Exchange — A New Hope. In *Proceedings of the 25th USENIX Security Symposium (USENIX Security 16)*, pages 327–343, Austin, TX, USA, 2016. USENIX Association.
 - [27] Joppe Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the Ring! Practical, Quantum-Secure Key Exchange from LWE. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1006–1018. ACM, 2016.
 - [28] Ronald Cramer, Léo Ducas, Chris Peikert, and Oded Regev. Recovering Short Generators of Principal Ideals in Cyclotomic Rings. Cryptology ePrint Archive, Paper 2015/313, 2015.

- [29] Han Wu and Guangwu Xu. Enhancing the Dual Attack against MLWE: Constructing More Short Vectors Using Its Algebraic Structure. Cryptology ePrint Archive, Paper 2022/1661, 2022.
- [30] Claus-Peter Schnorr and Martin Euchner. Lattice Basis Reduction: Improved Practical Algorithms and Solving Subset Sum Problems. *Mathematical Programming*, 66(1–3):181–199, 1994.
- [31] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better Lattice Security Estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2011.
- [32] Scott Fluhrer. Cryptanalysis of ring-LWE based key exchange with key share reuse. Cryptology ePrint Archive, Paper 2016/085, 2016.
- [33] Qian Guo, Erik Mårtensson, and Adrian Åström. The Perils of Limited Key Reuse: Adaptive and Parallel Mismatch Attacks with Post-processing Against Kyber. *IACR Communications in Cryptology*, 1(3), 2024.
- [34] Yue Qin, Chi Cheng, Xiaohan Zhang, Yanbin Pan, Lei Hu, and Jintai Ding. A Systematic Approach and Analysis of Key Mismatch Attacks on Lattice-Based NIST Candidate KEMs. Cryptology ePrint Archive, Paper 2021/123, 2021.
- [35] CRYSTALS-Kyber reference and optimized implementations. <https://github.com/pq-crystals/kyber>. Accessed: 2026-06-08.
- [36] Ronald Cramer, Léo Ducas, Chris Peikert, and Oded Regev. Recovering Short Generators of Principal Ideals in Cyclotomic Rings. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 559–585. Springer, 2016.
- [37] Ronald Cramer, Léo Ducas, and Benjamin Wesolowski. Short Stickelberger Class Relations and Application to Ideal-SVP. In *Advances in Cryptology – EUROCRYPT 2017*, volume 10210 of *Lecture Notes in Computer Science*, pages 324–348. Springer, 2017.
- [38] Jean-François Biasse and Fang Song. Efficient Quantum Algorithms for Computing Class Groups and Solving the Principal Ideal Problem in Arbitrary Degree Number Fields. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2016)*, pages 893–902. SIAM, 2016.
- [39] Martin R. Albrecht and Amit Deo. Large Modulus Ring-LWE $\geq$ Module-LWE. Cryptology ePrint Archive, Paper 2017/612, 2017.
- [40] Léo Ducas, Lynn Engelberts, and Paola de Perthuis. Predicting Module-Lattice Reduction. Cryptology ePrint Archive, Paper 2025/1904, 2025.

- [41] Paola de Perthuis and Filip Trenkić. Refined Modelling of the Primal Attack, and Variants Against Module-Learning With Errors. Cryptology ePrint Archive, Paper 2025/2195, 2025.
- [42] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of Learning with Errors. Cryptology ePrint Archive, Paper 2015/046, 2015.
- [43] Yu Wei, Lei Bi, Xianhui Lu, and Kunpeng Wang. Memory-Efficient BKW Algorithm for Solving the LWE Problem. Cryptology ePrint Archive, Paper 2025/266, 2025.
- [44] Koen de Boer and Wessel van Woerden. Lattice-based Cryptography: A Survey on the Security of the Lattice-based NIST Finalists. Cryptology ePrint Archive, Paper 2025/304, 2025.
- [45] Guillaume Hanrot, Xavier Pujol, and Damien Stehlé. Terminating BKZ. Cryptology ePrint Archive, Paper 2011/198, 2011.
- [46] Martin R. Albrecht, Léo Ducas, Gottfried Herold, Elena Kirshanova, Eamonn W. Postlethwaite, and Marc Stevens. The General Sieve Kernel and New Records in Lattice Reduction. Cryptology ePrint Archive, Paper 2019/089, 2019.
- [47] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. Cryptology ePrint Archive, Paper 2015/1128, 2015.
- [48] Thijs Laarhoven. *Search Problems in Cryptography: From Fingerprinting to Lattice Sieving*. PhD thesis, Eindhoven University of Technology, 2016.
- [49] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck. Estimating Quantum Speedups for Lattice Sieves. Cryptology ePrint Archive, Paper 2019/1161, 2019.
- [50] Max Heiser. Improved Quantum Hypercone Locality Sensitive Filtering in Lattice Sieving. Cryptology ePrint Archive, Paper 2021/1295, 2021.
- [51] André Chailloux and J. Loyer. Lattice Sieving via Quantum Random Walks. In Mihir Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021 – 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 63–91. Springer, 2021.
- [52] Thomas Espitau, Antoine Joux, and Natalia Kharchenko. On a Hybrid Approach to Solve Small Secret LWE, 2020.

- [53] Martin R. Albrecht. On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL. Cryptology ePrint Archive, Paper 2017/047, 2017.
- [54] Jung Hee Cheon, Minki Hhan, Seungwan Hong, and Yongha Son. A Hybrid of Dual and Meet-in-the-Middle Attack on Sparse and Ternary Secret LWE. *IACR Cryptology ePrint Archive*, 2019:1114, 2019.
- [55] Martin R. Albrecht et al. Lattice-Estimator. <https://github.com/malb/lattice-estimator>, 2015–. Accessed: 2026-03-24.
- [56] Nicolas Gama and Phong Q. Nguyen. Predicting Lattice Reduction. In Nigel Smart, editor, *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 31–51, Berlin, Heidelberg, 2008. Springer.
- [57] MATZOV. Report on the Security of LWE: Improved Dual Lattice Attack. Report, Zenodo, April 2022.
- [58] Léo Ducas. Shortest Vector from Lattice Sieving: A Few Dimensions for Free. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, volume 10820 of *Lecture Notes in Computer Science*, pages 125–145, Cham, 2018. Springer.
- [59] Qian Guo and Thomas Johansson. Faster Dual Lattice Attacks for Solving LWE with Applications to CRYSTALS. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, volume 13093 of *Lecture Notes in Computer Science*, pages 33–62, Cham, 2021. Springer.
- [60] Léo Ducas and Ludo N. Pulles. Does the Dual-Sieve Attack on Learning with Errors Even Work? In *Advances in Cryptology – CRYPTO 2023*, volume 14083 of *Lecture Notes in Computer Science*, pages 37–69, Cham, 2023. Springer.
- [61] Kévin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the Impact of a Variant of MATZOV’s Dual Attack on Kyber. In *Advances in Cryptology – CRYPTO 2025*, volume 16000 of *Lecture Notes in Computer Science*, pages 444–476, Cham, 2025. Springer.
- [62] TU Darmstadt Lattice Challenge. Svp challenge. <https://www.latticechallenge.org/svp-challenge/>, 2026. Accessed: 2026-03-26.
- [63] Florian Mendel, Tomislav Nad, and Martin Schläffer. Finding Collisions for Round-Reduced SM3, 2012.
- [64] Jian Zou, Wenling Wu, Shuang Wu, Bozhan Su, and Le Dong. Preimage Attacks on Step-Reduced SM3 Hash Function. In *Information Security and Cryptology – ICISC 2011*, 2012.

- [65] Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. On the Impact of Decryption Failures on the Security of LWE/LWR Based Schemes. IACR Cryptology ePrint Archive, Report 2018/1089, 2018.
- [66] Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption Failure Attacks on IND-CCA Secure Lattice-Based Schemes. In *Public-Key Cryptography – PKC 2019*, volume 11443 of *Lecture Notes in Computer Science*, pages 565–598. Springer, 2019.
- [67] Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. (One) Failure Is Not an Option: Bootstrapping the Search for Failures in Lattice-Based Encryption Schemes. In *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2020.
- [68] Jan-Pieter D’Anvers and Senne Batsleer. Multitarget Decryption Failure Attacks and their Application to Saber and Kyber. IACR Cryptology ePrint Archive, Report 2021/193, 2021.
- [69] Lipeng Wan, Fangyu Zheng, Guang Fan, Rong Wei, Lili Gao, Yuewu Wang, Jingqiang Lin, and Jiankuo Dong. A novel high-performance implementation of CRYSTALS-Kyber with AI accelerator. In *European Symposium on Research in Computer Security*, pages 514–534. Springer, 2022.
- [70] Xinyi Ji, Jiankuo Dong, Tonggui Deng, Pinchang Zhang, Jiafeng Hua, and Fu Xiao. HI-Kyber: A novel high-performance implementation scheme of Kyber based on GPU. *IEEE Transactions on Parallel and Distributed Systems*, 35(6):877–891, 2024.
- [71] Xuan Yu, Jipeng Zhang, Junhao Huang, Donglong Chen, and Lu Zhou. Multi-way High-Throughput Implementation of Kyber. In *International Conference on Information Security*, pages 41–60. Springer, 2024.
- [72] Jipeng Zhang, Yuxing Yan, Junhao Huang, and Çetin Kaya Koç. Optimized software implementation of Keccak, Kyber, and Dilithium on RV {32, 64} IM {B}{V}. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):632–655, 2025.
- [73] DongHyun Shin, YoungBeom Kim, Ayesha Khalid, Maire O’Neill, and Seog Chung Seo. Optimized Implementations of Keccak, Kyber, and Dilithium on the MSP430 Microcontroller. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2026(2):409–440, 2026.
- [74] Leon Botros, Matthias J Kannwischer, and Peter Schwabe. Memory-efficient high-speed implementation of Kyber on Cortex-M4. In *International Conference on Cryptology in Africa*, pages 209–228. Springer, 2019.

- [75] Denisa O. C. Greconici, Matthias J. Kannwischer, and Amber Sprenkels. Compact Dilithium Implementations on Cortex-M3 and Cortex-M4. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 1–24, 2021.
- [76] Joppe W Bos, Joost Renes, and Amber Sprenkels. Dilithium for Memory Constrained Devices. In *International Conference on Cryptology in Africa*, pages 217–235. Springer, 2022.

A Technical Proofs

Lemma 2.1. Let $q > 4$ be an odd integer and ℓ be a positive integer such that $2^{\ell+1} \mid (q-1)$. If x is chosen uniformly random from $\mathbb{Z}_q$, then the reconciliation function $\text{Rec}(x, w)$ defined as above is uniformly random conditioned on the signal $\text{Sig}(x) = w \in \{0, 1, \dots, 2^\ell - 1\}$.

Proof of Lemma 2.1. The following proof is valid for $2^{\ell+2} \mid (q-1)$.

Denote

- $J_i = \{y \in \mathbb{Z}_q : P(\text{Sig}(y) = i) > 0\}$,
- $p_i^- = \min_j \{j \in I_i\}$, for $i \neq 2^{\ell-1}$. For that last case, $p_{2^{\ell-1}}^- = \min_j \{j \in I_{2^{\ell-1}}, j > 0\}$,
- $p_i^+ = p_{i+1}^-$,
- $I'_i = I_i - \{p_i^-\}$,

so we can write:

$$J_i = \{p_i^-\} \dot{\cup} I'_i \dot{\cup} \{p_i^+\}.$$

and denote J_i^{even} (J_i^{odd}) the set of elements in J_i that are even (odd). Also, for any (but fixed) $y \in I'_i$, denote

$$\phi_i = P_{x \leftarrow \mathbb{Z}_q}(x = y \mid \text{Sig}(x) = i).$$

Notice that (by the randomized definition of Sig):

$$P_{x \leftarrow \mathbb{Z}_q}(x = p_i^- \mid \text{Sig}(x) = i) = P_{x \leftarrow \mathbb{Z}_q}(x = p_i^+ \mid \text{Sig}(x) = i) = \phi_i/2.$$

Now we distinguish cases. For $i = 0$:

$$J_0 = \underbrace{(\{p_0^-\} \dot{\cup} I_0^{\text{even}})}_{L/2+1 \text{ elements}} \dot{\cup} \underbrace{(\{p_0^+\} \dot{\cup} I_0^{\text{odd}})}_{L/2+1 \text{ elements}}.$$

For $i = 1, \dots, 2^{\ell-1} - 1$, $|I_i^{\text{even}}| = |I_i^{\text{odd}}|$, but $|I_i^{\text{even}}| = |I_i^{\text{odd}}| + 1$ and:

$$J_i = \underbrace{(I_i^{\text{even}})}_{L/2 \text{ elements}} \dot{\cup} \underbrace{(\{p_i^-, p_i^+\} \dot{\cup} I_i^{\text{odd}})}_{1+1+(L/2-1) \text{ elements}}$$

then

$$P_{x \leftarrow \mathbb{Z}_q}(\text{Rec}(x, i) = 0 \mid \text{Sig}(x) = i) = P_{x \leftarrow \mathbb{Z}_q}(x \in I_i^{\text{even}} \mid \text{Sig}(x) = i) = \phi_i \cdot L/2$$

and

$$P_{x \leftarrow \mathbb{Z}_q}(\text{Rec}(x, i) = 1 \mid \text{Sig}(x) = i) = P_{x \leftarrow \mathbb{Z}_q}(x \in \{p_i^-, p_i^+\} \dot{\cup} I_i^{\text{odd}} \mid \text{Sig}(x) = i) =$$

$$= \phi_i/2 + \phi_i/2 + (L/2 - 1)\phi_i = \phi_i L/2.$$

Similarly, for $i = 2^{\ell-1} + 1, \dots, 2^\ell - 1$, we only need to take into account that evens and odds are inverted, and that the parity is changed when taking the reconciliation function. Thus, $|I_i^{\text{even}}| = |I_i^{\text{odd}}|$, but $|I_i^{\text{odd}}| = |I_i^{\text{even}}| + 1$ and:

$$J_i = \underbrace{\left(I_i^{\text{odd}} \right)}_{L/2 \text{ elements}} \dot{\cup} \underbrace{\left(\{p_i^-, p_i^+\} \dot{\cup} I_i^{\text{even}} \right)}_{1+1+(L/2-1) \text{ elements}},$$

then

$$\begin{aligned} P_{x \leftarrow \mathbb{Z}_q}(\text{Rec}(x, i) = 0 \mid \text{Sig}(x) = i) &= P_{x \leftarrow \mathbb{Z}_q}(x \in \{p_i^-, p_i^+\} \dot{\cup} I_i^{\text{odd}} \mid \text{Sig}(x) = i) = \\ &= \phi_i/2 + \phi_i/2 + (L/2 - 1)\phi_i = \phi_i L/2 \end{aligned}$$

and

$$P_{x \leftarrow \mathbb{Z}_q}(\text{Rec}(x, i) = 1 \mid \text{Sig}(x) = i) = P_{x \leftarrow \mathbb{Z}_q}(x \in I_i^{\text{even}} \mid \text{Sig}(x) = i) = \phi_i \cdot L/2.$$

Finally, for $i = 2^{\ell-1}$, we just need to take into account that only the parity of half of the elements in $I_{2^{\ell-1}}$ is changed after applying the reconciliation function. Thus, denote $I_{2^{\ell-1}}^{\text{odd}+}$ ($I_{2^{\ell-1}}^{\text{odd}-}$) the odd positive (negative) elements in the interval, and act similarly for the even ones. We then have:

$$J_{2^{\ell-1}} = \underbrace{\left(I_{2^{\ell-1}}^{\text{odd}-} \dot{\cup} I_{2^{\ell-1}}^{\text{even}+} \right)}_{L/2 \text{ elements}} \dot{\cup} \underbrace{\left(\{p_{2^{\ell-1}}^-, p_{2^{\ell-1}}^+\} \dot{\cup} I_{2^{\ell-1}}^{\text{even}-} \dot{\cup} I_{2^{\ell-1}}^{\text{odd}+} \right)}_{1+1+(L/2-1) \text{ elements}},$$

then

$$P_{x \leftarrow \mathbb{Z}_q}(\text{Rec}(x, 2^{\ell-1}) = 0 \mid \text{Sig}(x) = 2^{\ell-1}) = P_{x \leftarrow \mathbb{Z}_q}(x \in I_{2^{\ell-1}}^{\text{even}+} \dot{\cup} I_{2^{\ell-1}}^{\text{odd}-} \mid \text{Sig}(x) = 2^{\ell-1}) = \phi_{2^{\ell-1}} \cdot L/2$$

and

$$\begin{aligned} P_{x \leftarrow \mathbb{Z}_q}(\text{Rec}(x, 2^{\ell-1}) = 1 \mid \text{Sig}(x) = 2^{\ell-1}) &= P_{x \leftarrow \mathbb{Z}_q}(x \in \{p_{2^{\ell-1}}^-, p_{2^{\ell-1}}^+\} \dot{\cup} I_{2^{\ell-1}}^{\text{odd}+} \dot{\cup} I_{2^{\ell-1}}^{\text{even}-} \mid \text{Sig}(x) = 2^{\ell-1}) = \\ &= \phi_{2^{\ell-1}}/2 + \phi_{2^{\ell-1}}/2 + (L/2 - 1)\phi_{2^{\ell-1}} = \phi_{2^{\ell-1}} L/2. \end{aligned}$$

Note that for the case $q - 1 = 2^{\ell+1}$ the proof is similar, but in that case *odd* and *even* must be interchanged at some points since $L = 2$. $\square$

Lemma 3.1. Let $k_A, k_B \in R_q$, let $\mathbf{w} = \text{Sig}(k_B)$, and let ℓ be the number of signal bits per coefficient. Denote $E = k_A - k_B$. If the inequality

$$\|E\|_\infty \leq \text{tol}_\ell := \frac{q-1}{2} \left(\frac{2^\ell - 1}{2^\ell} \right) - 1$$

holds and E consists only on even coefficients, then $\text{Rec}(k_A, \mathbf{w}) = \text{Rec}(k_B, \mathbf{w})$.

Proof of Lemma 3.1. We prove the result in $\mathbb{Z}_q$. Let $x, y \in \mathbb{Z}_q$, $b \in \{0, 1\}$ and let $w \in \{0, \dots, 2^\ell - 1\}$ be such that $y - b - wL \pmod{q} \in I_0$, and suppose that $e = x - y$ is an even number. Firstly, notice that

$$y - wL \pmod{q} \in I_0 \cup \left\{ \frac{L}{2} + 1 \right\}$$

and consequently

$$|y - wL \pmod{q}| \leq \frac{L}{2} + 1. \quad (3)$$

Now,

$$x - wL \pmod{q} = y - wL + e \pmod{q}. \quad (4)$$

If

$$|y - wL \pmod{q}| + |e| \leq \frac{q-1}{2} \quad (5)$$

then we have

$$y - wL + e \pmod{q} = y - wL \pmod{q} + e$$

and consequently (using that e is an even number and identity 4)

$$y - wL + e \pmod{q} \pmod{2} = y - wL \pmod{q} \pmod{2}.$$

But using inequality 3, condition 5 is equivalent to

$$|e| \leq \frac{q-1}{2} \left(\frac{2^\ell - 1}{2^\ell} \right) - 1.$$

□

Proposition 3.2. The protocol described in 2.4 outputs equal shared secrets $ss_A = ss_B$ if, for the secret and error vectors sampled during a run of such protocol — namely $\mathbf{s}_A, \mathbf{e}_A, \mathbf{s}_B, \mathbf{e}_B, e_B$ —, the following inequation holds:

$$\|\mathbf{e}_A \cdot \mathbf{s}_B - (\mathbf{e}_B + \mathbf{r}_B) \cdot \mathbf{s}_A + e_B\|_\infty \leq \frac{\text{tol}_\ell}{2},$$

where $\mathbf{r}_B := \mathbf{u}_B'' - \mathbf{u}_B$, and tol_ℓ is defined as in Lemma 3.1.

Proof of Proposition 3.2. The key point is that in this case $k_A - k_B = 2 \cdot (\mathbf{e}_A \cdot \mathbf{s}_B - (\mathbf{e}_B + \mathbf{r}_B) \cdot \mathbf{s}_A + e_B)$. Then we simply apply Lemma 3.1. □

B Parameter selection

Parameters ($q = 3329, n = 256$)				Failure prob.	CC bytes		CC bytes ML-KEM	Classical sec. bits (malb)		Sym. (NIST)	sec bits (NGCC)
k	η	d_B ($d_A = 12$)	ℓ		CPA	CCA		Primal	Dual hybrid		
2	3	10	4	-132.7	1568	1600	1568	142.1	139.7	AES128 (143 bits)	128
			3	-115.9	1536	1568					
		9	4	-78.8	1504	1536					
4	2	11	5	-181.2	3136	3168	3136	275.4	262.3	AES 256 (272 bits)	256
			4	-169.8	3104	3136					
			3	-147.9	3072	3104					
		10	5	-147.6	3008	3040					

Table 8: Some possible parameter choices for the first two security levels.

Parameters							Failure prob	CC bytes		Classical security bits	
q	n	k	η	ℓ	d_A	d_B		CPA	CCA	Primal	Dual hybrid
3329	256	8	2	4	12	12	-101.9	6304	6336	571.1	537.6
				6	12	12	-112.8	6368	6400		
				8	12	12	-115.3	6432	6464		
7681	512	4	2	2	13	13	-349.8	6848	6912	526.6	495.5
				2	13	12	-283.3	6592	6656		
				2	13	11	-209.3	6336	6400		
				4	13	10	-160.1	6208	6272		
			3	4	13	13	-242.2	6976	7040	552.9	519.9
				4	13	12	-209.3	6720	6784		
				6	13	11	-184.3	6592	6656		
				4	13	11	-167.0	6464	6528		
12289	512	4	3	2	14	12	-323.1	6848	6912	527.8	496.9
				2	14	11	-192.6	6592	6656		
				6	14	10	-134.4	6592	6656		
			4	3	14	13	-279.0	7168	7232	546.2	514.1
				2	14	13	-205.4	7104	7168		
				3	14	12	-259.3	6912	6976		
				3	14	11	-168.7	6656	6720		

Table 9: Some possible (preliminary) parameter choices for the third security level.